

Question 1

Difficulty: Beginner

A developer is ingesting plain-text product documentation into a Databricks RAG pipeline. The embedding model they chose has a maximum context limit of 512 tokens. Which chunking concern is MOST critical to address first?

- **A)** The chunks must all contain exactly the same number of sentences so that the LLM receives uniformly structured context regardless of the length of each sentence in the original document.
- **B)** No chunk should exceed 512 tokens, because text beyond the token limit is silently truncated by the embedding model, causing information loss and degrading the quality of the resulting vector.
- **C)** Every chunk must begin with the document's title so the LLM always knows which source document a chunk belongs to, preventing it from confusing content from different product manuals.
- **D)** Each chunk must be stored as a separate file in a Databricks Volume before embedding, because the embedding model cannot process chunks passed as in-memory Python strings directly.

Correct Answer: B **Explanation:** B is correct. Embedding models have a hard token limit; any text beyond that limit is truncated before the embedding is computed. This means part of the chunk's meaning is permanently lost, producing a misleading embedding vector. Ensuring no chunk exceeds 512 tokens is the most fundamental constraint when working with this model. A is wrong because uniform sentence count is not a requirement — chunk quality is about semantic completeness within the token limit, not uniform length. C is wrong because while adding metadata (like document title) is a useful best practice, it is a secondary concern to first ensuring chunks fit the model's token limit. D is wrong because embedding models accept text strings directly in API calls; writing each chunk to a file first is unnecessary overhead with no benefit. **Source:** Section 2: Data Preparation – Objective 1: Apply a chunking strategy for a given document structure and model constraints — docs.databricks.com (search: "Data preparation for RAG applications")

Question 2

Difficulty: Beginner

A data engineer is preparing HTML web pages for a RAG knowledge base. Each page contains the actual article text, but also a navigation bar, header logo text, footer with copyright notices, and cookie consent banners. What should be done to these extra elements before chunking?

- **A)** Include all HTML elements in the chunks unchanged, because the embedding model and LLM are trained on web data and can automatically distinguish navigation text from article content.
- **B)** Replace the navigation, footer, and banner text with placeholder tokens like `[NAV]` , `[FOOTER]` , and `[BANNER]` so the LLM knows to ignore them when generating answers.
- **C)** Filter and remove the navigation bar, header, footer, and cookie banners before chunking, so that only the meaningful article content is embedded and stored in the Vector Search index.
- **D)** Convert all HTML pages to PDF format first, because PDF parsers automatically strip navigation elements and return only the main body text for downstream chunking and embedding.

Correct Answer: C **Explanation:** C is correct. Navigation bars, footers, and cookie banners are extraneous content — they add noise to the embedding without contributing relevant knowledge. Embedding these elements dilutes the semantic signal of the chunk and can cause the retriever to return irrelevant chunks that happen to match boilerplate phrases. Filtering them out using a library like `beautifulsoup4` before chunking is the standard approach. A is wrong because LLMs and embedding models do not automatically filter out boilerplate HTML structure; they treat all ingested text as meaningful content, and repeated boilerplate degrades retrieval quality. B is wrong because placeholder tokens still consume embedding space and teach the retriever to associate queries with boilerplate metadata, which is counterproductive. D is wrong because PDF conversion does not automatically remove navigation elements; HTML-to-PDF tools typically include all visible page elements, and the conversion adds processing overhead without solving the filtering problem. **Source:** Section 2: Data Preparation – Objective 2: Filter extraneous content in source documents — docs.databricks.com (search: "Generative AI data preparation")

Question 3

Difficulty: Beginner

A team has a folder of scanned paper documents (image-based PDFs where the text is not selectable) and needs to extract the text content for a RAG pipeline. Which Python library should they use for this task?

- **A)** `beautifulsoup4` — because it can parse any binary file format including image-based PDFs by treating the file as an HTML document and stripping non-text elements.
- **B)** `PyPDF2` — because it reads PDF files page by page and extracts embedded text, including text in rasterized image layers within the PDF file.
- **C)** `pytesseract` — because it performs Optical Character Recognition (OCR) on images and scanned documents, converting the visual text in the scanned pages into machine-readable strings.
- **D)** `unstructured` — because it natively handles all file formats including scanned image PDFs by directly reading the binary pixel data and converting it to structured text elements.

Correct Answer: C **Explanation:** C is correct. `pytesseract` is Python's wrapper for Google's Tesseract OCR engine. It processes images and scanned PDFs by analyzing the pixel patterns of characters and converting them into text strings. This is the only option that can extract text from image-based content where no digital text layer exists. A is wrong because `beautifulsoup4` is exclusively for parsing HTML/XML structured text — it has no capability to process binary image data or perform OCR. B is wrong because `PyPDF2` can only extract digitally embedded text layers from PDFs — it cannot read text from rasterized image layers (scanned pages), where no text layer exists. D is wrong because while `unstructured` is a powerful multi-format library, it relies on OCR tools (including Tesseract) as a backend for scanned PDFs; `pytesseract` is the direct tool that actually performs the OCR operation. **Source:** Section 2: Data Preparation – Objective 3: Choose the appropriate Python package to extract document content — [docs.databricks.com](https://docs.databricks.com/search/rag-reference-architecture-data-preparation) (search: "RAG reference architecture data preparation")

Question 4

Difficulty: Beginner

After chunking product documents, a developer has a Spark DataFrame with columns `id` (unique chunk identifier) and `chunk_text` (the text content). What is the correct sequence of operations to prepare this data for Databricks Vector Search?

- **A)** Create the Vector Search index first, then write the DataFrame to a Delta table, then enable Change Data Feed (CDF) on the table so that future updates sync to the index automatically.
- **B)** Write the DataFrame to a Unity Catalog Delta table, enable Change Data Feed (CDF) on the table with `SET TBLPROPERTIES (delta.enableChangeDataFeed = true)`, then create the Vector Search index pointing to that table.
- **C)** Enable CDF on the Unity Catalog schema first, then write the DataFrame to a Delta table, then create a Vector Search endpoint, and finally create the index using the endpoint name as the table reference.
- **D)** Write the DataFrame to a Databricks Volume as JSON files, enable Change Data Feed on the volume folder, then create a Vector Search index pointing to the volume path as the data source.

Correct Answer: B **Explanation:** B is correct. The required sequence is: (1) write chunked data to a Unity Catalog Delta table, (2) enable Change Data Feed on that table (required for Vector Search to detect and sync incremental updates), (3) create the Vector Search index pointing to the Delta table. CDF must be enabled before the index is created so Vector Search can establish the sync pipeline. A is wrong because the index cannot be created before the source table exists and has CDF enabled — Vector Search needs to read the existing table data and set up CDF-based sync at index creation time. C is wrong because CDF is a table-level property, not a schema-level setting; you enable it per-table, not per-schema. D is wrong because Databricks Vector Search requires a Delta Lake table as its source — it does not support Volume JSON files as a sync source, and Volumes do not have Change Data Feed. **Source:** Section 2: Data Preparation – Objective 4: Define operations and sequence to write chunked text into Delta Lake — docs.databricks.com (search: "Create a Mosaic AI Vector Search index" and "Enable Change Data Feed")

Question 5

Difficulty: Beginner

A company is building an IT Support chatbot on Databricks. They have three potential document sources: (A) current approved troubleshooting guides (updated quarterly), (B) legacy system manuals from 2008 that cover deprecated hardware, and (C) employee birthday party announcements from the internal newsletter. Which documents should be included in the RAG knowledge base?

- **A)** All three sources should be included to maximize the breadth of the knowledge base; the LLM will automatically determine which content is relevant and ignore the irrelevant sources at query time.
- **B)** Only source A (current troubleshooting guides) should be included, as it is the only source containing accurate, current, task-relevant information that the chatbot is expected to answer questions about.
- **C)** Sources A and B should be included because the legacy manuals may still contain relevant troubleshooting principles even if the specific hardware is deprecated, giving the chatbot more context depth.
- **D)** Sources A and C should be included because current guides answer technical questions, and newsletter announcements help the chatbot maintain a friendly conversational tone in its responses.

Correct Answer: B **Explanation:** B is correct. The knowledge base should contain only documents that are accurate, current, and directly relevant to the chatbot's purpose (IT support). The 2008 legacy manuals (B) contain outdated information about deprecated hardware that will produce incorrect or misleading answers. The birthday announcements (C) are entirely unrelated and will degrade retrieval precision by matching queries about people's names or dates. Including only source A ensures the highest quality knowledge base. A is wrong because LLMs cannot reliably "ignore" irrelevant retrieved content — they tend to incorporate whatever context they receive, including outdated or irrelevant information, into their generated answers. C is wrong because including deprecated hardware manuals is likely to cause the chatbot to give outdated troubleshooting steps for hardware the company no longer uses. D is wrong because newsletter announcements contain no IT support knowledge; including them would cause the retriever to return birthday announcements as context for technical queries. **Source:** Section 2: Data Preparation – Objective 5: Identify needed source documents — docs.databricks.com (search: "Generative AI data preparation best practices")

Question 6

Difficulty: Intermediate

A developer has chunked a Markdown-formatted technical reference guide using fixed-size chunking with 500 characters and 50-character overlap. Users report that answers about configuration options are incomplete. Analysis shows that configuration tables in the Markdown are being split mid-row across chunk boundaries. What is the most targeted fix?

- **A)** Increase the chunk size from 500 to 5,000 characters, which ensures that even the longest configuration tables fit within a single chunk and are never split across boundaries.
- **B)** Switch from fixed-size chunking to document-aware (structure-based) chunking using Markdown headers and table boundaries as split points, keeping configuration tables intact within a single chunk.
- **C)** Add a 200-character overlap instead of 50 characters so that when a table is split, the overlapping content on both sides reconstructs the missing rows for the LLM during retrieval.
- **D)** Convert the Markdown tables to plain prose before chunking (e.g., "Option X has value Y") so that the fixed-size chunker can split the flattened text without breaking logical table rows.

Correct Answer: B **Explanation:** B is correct. The root cause is that fixed-size chunking is structure-agnostic — it splits by character count regardless of document structure, breaking tables mid-row. Document-aware chunking uses the document's own structure (Markdown headers `##`, horizontal rules, table boundaries) as natural split points. A library like LangChain's `MarkdownHeaderTextSplitter` or `unstructured` can detect these boundaries and keep tables intact within a chunk. A is wrong because increasing chunk size to 5,000 characters is a blunt fix that may solve the immediate problem but creates very large chunks that exceed embedding model token limits and reduce retrieval precision. C is wrong because overlap preserves context at the edge of chunks, but does not reconstruct full table rows; a 200-character overlap would duplicate the row header but not the missing data rows. D is wrong because converting tables to prose is a complex transformation that may lose the relational structure of the table data, making it harder for the LLM to reason about comparative configuration values.

Source: Section 2: Data Preparation – Objective 7: Design retrieval systems using advanced chunking strategies — docs.databricks.com (search: "Advanced chunking strategies RAG")

Question 7

Difficulty: Intermediate

After deploying a RAG application on Databricks, a developer wants to measure whether the Vector Search retriever is returning the correct document chunks for a given set of test queries. They have a test dataset with queries and the known correct chunk IDs that should be retrieved. Which approach uses the correct Databricks tooling?

- **A)** Manually compare the retrieved chunk IDs to the expected IDs in a Python loop, calculate a match percentage, and log the result as a custom MLflow metric using `mlflow.log_metric("retrieval_accuracy", score)`.
- **B)** Decorate the retrieval function with `@mlflow.trace(span_type="RETRIEVER")`, then run `mlflow.genai.evaluate()` with the `RetrievalRelevance` scorer, which uses the captured trace data to evaluate retrieval quality.
- **C)** Use Databricks Model Serving's built-in A/B testing framework to compare two retriever configurations and let the platform automatically select the one with higher user click-through rate on retrieved results.
- **D)** Query the Vector Search index directly using the REST API, collect the returned chunk IDs into a Delta table, and then run a Spark JOIN against the expected results table to calculate Precision@k.

Correct Answer: B Explanation: B is correct. The Databricks-recommended approach for evaluating retrieval is to instrument the retriever with `@mlflow.trace(span_type="RETRIEVER")` so MLflow captures exactly which chunks were retrieved for each query. Then `mlflow.genai.evaluate()` with the `RetrievalRelevance` scorer uses an LLM judge to assess whether the retrieved chunks are relevant to the query. This integrates seamlessly with MLflow's evaluation framework. A is wrong because manually logging a custom match percentage works but is not the Databricks-recommended approach — it bypasses MLflow's built-in GenAI evaluation framework and provides a less insightful metric than `RetrievalRelevance`. C is wrong because A/B testing with click-through rate is a user behavioral metric, not a retrieval quality metric — it requires actual users, not a test dataset, and measures user engagement, not chunk relevance. D is wrong because manually running a Spark JOIN to calculate Precision@k works as a data engineering approach but bypasses the MLflow evaluation framework and requires significant custom code compared to the native `mlflow.genai.evaluate()` approach.

Source: Section 2: Data Preparation – Objective 6: Use tools and metrics to evaluate retrieval performance — docs.databricks.com (search: "MLflow evaluate for RAG" and "MLflow Tracing")

Question 8

Difficulty: Intermediate

A RAG application retrieves the top-5 chunks ($k=5$) from Vector Search for each query. A developer wants to understand what `Recall@5` measures in this context and how to interpret a value of 0.6.

- **A)** `Recall@5` measures what fraction of the 5 retrieved chunks are relevant (relevant retrieved $\div$ total retrieved), so 0.6 means 3 out of 5 retrieved chunks are relevant to the query.
- **B)** `Recall@5` measures what fraction of ALL known relevant chunks were found in the top-5 results (relevant retrieved $\div$ total relevant), so 0.6 means 60% of all relevant chunks exist in the top-5 results.
- **C)** `Recall@5` measures the average rank position of the first relevant chunk within the top-5 results, so 0.6 means the first relevant chunk appears at rank position 3 on a normalized 0-to-1 scale.
- **D)** `Recall@5` measures the semantic similarity score of the 5th-ranked chunk to the query, so 0.6 means the least-relevant chunk in the top-5 has a cosine similarity of 0.6 to the query vector.

Correct Answer: B **Explanation:** B is correct. $\text{Recall}@k = (\text{number of relevant chunks retrieved in top-}k) \div (\text{total number of relevant chunks in the dataset})$. A $\text{Recall}@5$ of 0.6 means that out of all the chunks that are actually relevant to the query (say, 5 relevant chunks exist total), 60% of them (3 chunks) appear in the top-5 retrieved results. High recall means you're not missing relevant information. A is wrong because that definition describes $\text{Precision}@k$, not $\text{Recall}@k$. $\text{Precision}@5 = (\text{relevant retrieved}) \div k$ (total retrieved = 5). C is wrong because that describes Mean Reciprocal Rank (MRR), which measures the rank position of the first relevant result. D is wrong because that describes a raw similarity threshold, not a retrieval recall metric; $\text{Recall}@k$ is about coverage of relevant documents, not individual similarity scores. **Source:** Section 2: Data Preparation – Objective 6: Use tools and metrics to evaluate retrieval performance — docs.databricks.com (search: "MLflow evaluate for RAG")

Question 9

Difficulty: Intermediate

A developer needs to extract text from Word documents (`.docx`), HTML web pages, and regular text-based PDFs in a single unified pipeline on Databricks. They want a single Python library that handles all three formats and returns structured output (separating titles from body text). Which library is most appropriate?

- **A) PyPDF2** — because it supports cross-format parsing and automatically detects the file type from the file extension, switching between PDF, HTML, and Word parsing modes internally.
- **B) pytesseract** — because it uses the underlying image rendering of all document types (converting each format to an image first) to extract text uniformly across PDFs, HTML, and Word documents.
- **C) unstructured** — because it is a multi-format document parsing library that handles PDFs, Word, HTML, and many other formats, returning structured elements (titles, narrative text, tables) from each format.
- **D) BeautifulSoup4** — because it is format-agnostic and can parse the raw binary content of any file type, automatically recognizing Word, HTML, and PDF document structures.

Correct Answer: C **Explanation:** C is correct. `unstructured` is the go-to library for multi-format document ingestion in RAG pipelines. It natively supports `.docx`, `.html`, `.pdf`, and many other formats, and parses them into typed elements (e.g., `Title`, `NarrativeText`, `Table`) — making it ideal for pipelines that need both format flexibility and structured output. A is wrong because `PyPDF2` only handles PDF files; it has no capability to parse Word or HTML documents. B is wrong because `pytesseract` is an OCR library for image-based text; standard Word, HTML, and digital PDFs have embedded text layers that do not need OCR, and `pytesseract` would be unnecessarily slow and lossy for these formats. D is wrong because `BeautifulSoup4` is exclusively an HTML/XML parser; it cannot parse Word documents or PDFs, and it does not process binary file formats. **Source:** Section 2: Data Preparation – Objective 3: Choose the appropriate Python package to extract document content — docs.databricks.com (search: "RAG reference architecture data preparation")

Question 10

Difficulty: Intermediate

A Databricks Vector Search index is configured with `pipeline_type="TRIGGERED"` and source table `main.docs.policy_chunks`. A developer updates 500 rows in the source Delta table. When will these updates appear in the Vector Search index?

- **A)** Immediately — because Delta Lake's ACID transactions ensure that all Vector Search indexes always reflect the current committed state of the source table in real time.
- **B)** Within 5 seconds — because `TRIGGERED` mode uses Change Data Feed micro-batching with a default polling interval of 5 seconds to detect and apply changes from the source Delta table.
- **C)** Only after a manual sync is triggered — either by calling `index.sync()` via the Python SDK or scheduling a sync via the Databricks Vector Search UI or REST API, since `TRIGGERED` mode does not auto-sync.
- **D)** At the next scheduled Vector Search maintenance window — because Databricks performs index synchronization during off-peak hours to avoid impacting production query performance.

Correct Answer: C **Explanation:** C is correct. `TRIGGERED` pipeline mode means the Vector Search index only updates when explicitly triggered by calling `index.sync()` (Python SDK), the REST API, or via the UI. Updates to the source Delta table are detected via Change Data Feed, but they do not automatically propagate to the index — the developer must invoke a sync. This mode is suitable for batch workloads where real-time updates are not needed. A is wrong because Delta ACID transactions guarantee consistency at the Delta table level, not at the Vector Search index level; the index is a separate derived artifact that must be explicitly synchronized. B is wrong because micro-batch auto-polling is the behavior of `CONTINUOUS` pipeline mode, not `TRIGGERED` mode; these two modes have fundamentally different sync behaviors. D is wrong because there is no automatic maintenance window sync in Databricks Vector Search; all syncs in `TRIGGERED` mode are explicitly initiated by the developer.

Source: Section 2: Data Preparation – Objective 4: Define operations and sequence to write chunked text into Delta Lake — [docs.databricks.com](https://docs.databricks.com/search/create-mosaic-ai-vector-search-index.html) (search: "Create a Mosaic AI Vector Search index")

Question 11

Difficulty: Advanced

A developer applies fixed-size chunking with 512 tokens and 64-token overlap to a lengthy legal contract. They observe that the embeddings for consecutive chunks have very high cosine similarity (≥ 0.97), making it hard for the retriever to distinguish between adjacent chunks. What is the most likely root cause and the correct fix?

- **A)** The cosine similarity is high because the 64-token overlap introduces duplicate content in consecutive chunks; reducing the overlap to zero eliminates the shared content and creates more distinct embeddings.
- **B)** The cosine similarity is high because legal contracts use highly repetitive boilerplate language (e.g., "the party of the first part shall..."); document-aware chunking by contract section boundaries will produce more semantically distinct chunks.
- **C)** The cosine similarity is high because the embedding model has not been fine-tuned on legal text; replacing the embedding model with a legal-domain-specific model will produce embeddings that better distinguish contract clauses.
- **D)** The cosine similarity is high because 512 tokens is too small for legal text; increasing the chunk size to 2,048 tokens makes each chunk more contextually complete, reducing semantic overlap with neighboring chunks.

Correct Answer: B Explanation: B is correct. Legal contracts are densely repetitive — the same boilerplate phrases, party names, and legal constructions appear throughout. Fixed-size chunking cuts across this repeated structure arbitrarily, producing chunks that are semantically nearly identical because they share the same vocabulary and clause patterns. Document-aware chunking by contract section (e.g., "Section 4: Indemnification" as one chunk, "Section 5: Limitation of Liability" as another) produces semantically distinct chunks that are more differentiable. A is wrong because reducing overlap to zero reduces the duplicate content between adjacent chunks but does not solve the root cause — boilerplate legal language across all sections still produces high similarity regardless of overlap size. C is wrong because while a legal-domain embedding model may perform better overall, the root cause here is the structural repetitiveness of the source document, not the embedding model's domain knowledge. D is wrong because larger chunks include more context but the boilerplate density remains; you'd still get high similarity between large chunks filled with similar legal language. **Source:** Section 2: Data Preparation – Objective 1 & 7: Chunking strategy and advanced chunking — docs.databricks.com (search: "Advanced chunking strategies RAG")

Question 12

Difficulty: Advanced

A developer implements Parent-Child chunking for a RAG pipeline. Small "child" chunks (150 tokens) are embedded and stored in Vector Search. When a query matches a child chunk, the retriever fetches the corresponding "parent" chunk (1,000 tokens) to send to the LLM. What problem does this architecture solve compared to embedding large chunks directly?

- **A)** It reduces the total storage cost of the Vector Search index because only the small child embeddings are stored as vectors, while the large parent chunks remain as plain text in the Delta table without requiring vector storage.
- **B)** It solves the precision-context tradeoff: small child chunks produce precise, focused embeddings that match queries accurately (high precision retrieval), while the large parent chunks provide the LLM with sufficient surrounding context to generate a complete answer.
- **C)** It eliminates the need for Change Data Feed because parent chunks are static and never updated — only the child chunks require CDF-based synchronization with the Vector Search index.
- **D)** It improves retrieval speed because the Vector Search index only needs to store a fraction of the data (child chunks), reducing index size and making approximate nearest neighbor search significantly faster.

Correct Answer: B Explanation: B is correct. This is the exact problem Parent-Child chunking is designed to solve. Embedding large chunks directly produces "fuzzy" embeddings that mix many topics, making precise query matching harder. Small child chunks produce highly focused, precise embeddings that match specific query terms well. However, when the LLM receives only a 150-token child chunk as context, it often lacks enough surrounding information to generate a complete, well-grounded answer. Fetching the 1,000-token parent chunk gives the LLM rich context while maintaining high retrieval precision. A is wrong because while it is true that only child embeddings are vectorized, the primary motivation for Parent-Child chunking is retrieval precision and context richness, not storage cost reduction. C is wrong because CDF is required whenever the source Delta table is updated, regardless of whether parent or child chunks change — the sync requirement is determined by the table, not by chunk hierarchy. D is wrong because while a smaller index does improve search speed, this is a secondary benefit; the primary architectural motivation is the precision-context tradeoff described in B. **Source:** Section 2: Data Preparation – Objective 7: Design retrieval systems using advanced chunking strategies — docs.databricks.com (search: "Advanced chunking strategies RAG")

Question 13

Difficulty: Advanced

A developer uses `mlflow.genai.evaluate()` with a `RetrievalRelevance` scorer on a test dataset. The test queries are logged but the scorer returns all null scores. Investigation reveals the `@mlflow.trace(span_type="RETRIEVER")` decorator was not applied to the retrieval function. Why does this cause null scores, and what is the fix?

- **A)** Without the `RETRIEVER` span, MLflow's `RetrievalRelevance` scorer cannot identify which part of the trace corresponds to the retrieval step; it needs the typed span to know which inputs and outputs to pass to the LLM judge for scoring.
- **B)** Without the `RETRIEVER` span, the retrieval function runs synchronously instead of asynchronously, which causes a thread lock that prevents MLflow from recording any evaluation scores during the evaluation run.
- **C)** Without the `RETRIEVER` span, the evaluation function falls back to using the final LLM output instead of the retrieved chunks for scoring, which inflates the relevance scores to 1.0 rather than returning null.
- **D)** Without the `RETRIEVER` span, MLflow cannot authenticate the retrieval function's calls to the Vector Search index, causing a permissions error that silently returns null scores instead of raising an exception.

Correct Answer: A **Explanation:** A is correct. MLflow's `RetrievalRelevance` scorer is specifically designed to evaluate the output of the retrieval step — the documents that were fetched. It finds this data by looking for a trace span typed as `"RETRIEVER"`, which contains the query input and the retrieved documents as outputs. Without this typed span, MLflow cannot locate the retrieval step in the trace tree and has no data to pass to the LLM judge, resulting in null scores. The fix is to add `@mlflow.trace(span_type="RETRIEVER")` to the retrieval function. B is wrong because the `RETRIEVER` span type is metadata for MLflow's evaluation framework — it has no effect on whether the function runs synchronously or asynchronously. C is wrong because without a `RETRIEVER` span, MLflow doesn't fall back to using LLM output; it simply cannot find the data it needs and produces nulls. D is wrong because MLflow tracing and Vector Search authentication are independent; the `RETRIEVER` decorator is an instrumentation call, not an authentication mechanism. **Source:** Section 2: Data Preparation – Objective 6: Use tools and metrics to evaluate retrieval performance — docs.databricks.com (search: "MLflow Tracing")

Question 14

Difficulty: Advanced

A developer needs to enable Change Data Feed on a Delta table that already has 2 million existing rows before creating a Vector Search index. They run: `ALTER TABLE main.docs.chunks SET TBLPROPERTIES (delta.enableChangeDataFeed = true)`. Will the existing 2 million rows be synced to the Vector Search index when it is created?

- **A)** No — enabling CDF only captures future changes (inserts, updates, deletes) after the property is set. When the Vector Search index is created, it performs an initial full-table snapshot read of all existing rows to seed the index, then uses CDF for subsequent incremental updates.
- **B)** No — enabling CDF on an existing table has no effect on historical data; the Vector Search index will only contain rows that were inserted or updated AFTER CDF was enabled, leaving all 2 million existing rows out of the index.
- **C)** Yes — enabling CDF retroactively creates change records for all 2 million existing rows, which the Vector Search index reads during its first sync to populate the index with all historical data.
- **D)** Yes — but only if the developer runs `OPTIMIZE TABLE main.docs.chunks` after enabling CDF, which compacts the existing data files and triggers CDF to generate historical change records for all rows.

Correct Answer: A **Explanation:** A is correct. When a Vector Search index is created on a Delta table with CDF enabled, it performs an initial full-table snapshot read to seed the index with all existing rows. After this initial load, it uses CDF records to track only incremental changes (new inserts, updates, deletes) going forward. This means all 2 million existing rows are correctly included in the index. B is wrong because it confuses CDF's ongoing behavior (capturing changes) with the index creation behavior (initial full snapshot). The initial snapshot reads all rows regardless of when CDF was enabled. C is wrong because enabling CDF does not retroactively create CDF records for historical data — CDF records only begin from the transaction version when CDF is enabled. The existing rows are captured via the initial snapshot, not via CDF records. D is wrong because `OPTIMIZE` is for file compaction and does not generate CDF records for historical data; it is unrelated to the Vector Search index creation process. **Source:** Section 2: Data Preparation – Objective 4: Define operations and sequence to write chunked text into Delta Lake — docs.databricks.com (search: "Enable Change Data Feed" and "Create a Mosaic AI Vector Search index")

Question 15

Difficulty: Advanced

A developer is building a RAG pipeline for a company that publishes new product release notes weekly. The existing Vector Search index uses `pipeline_type="TRIGGERED"` sync. The team wants to ensure the index reflects new release notes within 1 hour of publication to the Delta table. What is the most operationally efficient solution?

- **A)** Switch the Vector Search index sync pipeline to `CONTINUOUS` mode, which automatically propagates Delta table changes to the index in near-real-time without requiring any scheduled jobs or manual intervention.
- **B)** Keep the `TRIGGERED` mode but deploy a Databricks Workflow job scheduled every 1 hour that calls `index.sync()` on the Vector Search index after new data is written to the source Delta table.
- **C)** Switch to `CONTINUOUS` mode and enable Delta Structured Streaming on the source table, which creates a push-based pipeline where new rows are streamed directly to the Vector Search index as they are committed.
- **D)** Keep `TRIGGERED` mode and configure a Databricks Alert on the source Delta table that sends an API call to `index.sync()` whenever new rows are detected, replacing the need for a scheduled job.

Correct Answer: B **Explanation:** B is correct. For a 1-hour freshness SLA, a scheduled Workflow job that calls `index.sync()` every hour is the most straightforward and operationally efficient solution with `TRIGGERED` mode. This approach is cost-effective (sync only runs when scheduled, not continuously) and meets the SLA. A is wrong because `CONTINUOUS` mode would also satisfy the SLA but is more expensive — it keeps the sync pipeline running continuously even when there are no changes, incurring ongoing compute cost for what is a weekly update pattern. C is wrong because `CONTINUOUS` mode uses CDF-based propagation internally — there is no separate "Delta Structured Streaming push" mechanism for Vector Search; the description conflates two different Databricks technologies. D is wrong because Databricks Alerts are monitoring notifications (email, Slack) triggered by SQL query conditions — they cannot directly call the `index.sync()` API; this would require a webhook integration, which is significantly more complex than a scheduled Workflow job. **Source:** Section 2: Data Preparation – Objective 4: Define operations and sequence to write chunked text into Delta Lake — [docs.databricks.com](https://docs.databricks.com/search/creating-vector-search-index.html) (search: "Create a Mosaic AI Vector Search index")

Question 16

Difficulty: Proficiency

A company ingests 10,000 long-form research papers (each averaging 8,000 tokens) into a RAG pipeline using fixed-size chunking with 512-token chunks and 50-token overlap. Retrieval quality (`Recall@5 = 0.41`) is low. An engineer proposes Parent-Child chunking as the fix. A second engineer argues that improving the embedding model is the right solution. How should the team evaluate which fix addresses the root cause?

- **A)** Run an ablation study: first replace the embedding model while keeping fixed-size chunking, then switch to Parent-Child chunking while keeping the original embedding model, and compare `Recall@5` for each change to determine which variable drives the improvement.
- **B)** Switch to Parent-Child chunking immediately because chunking strategy always has a larger impact on retrieval quality than embedding model selection for long-form documents, making further evaluation unnecessary.
- **C)** Replace the embedding model first because embedding quality is always the primary bottleneck in RAG pipelines; chunking strategy is a secondary factor that only matters after the optimal embedding model is selected.
- **D)** Add more documents to the knowledge base to increase the probability that the correct chunk is retrieved, since low `Recall@5` always indicates a knowledge gap in the source documents rather than a retrieval architecture problem.

Correct Answer: A **Explanation:** A is correct. The correct engineering approach is an ablation study — isolating each variable (embedding model vs. chunking strategy) while holding the other constant, and measuring the impact on `Recall@5`. This produces evidence about which factor is the actual bottleneck. For 8,000-token papers with 512-token chunks, both factors could be significant: large papers may have relevant content spread across many chunks (favoring Parent-Child), and a weak embedding model may fail to match query semantics (favoring model upgrade). B is wrong because there is no universal rule that chunking always dominates over embedding quality — the relative importance depends on the specific content type, query patterns, and current setup; assuming so without evidence is a logical error. C is wrong for the same reason as B but in the opposite direction — there is no universal hierarchy that embedding model always matters more than chunking. D is wrong because `Recall@5 = 0.41` means 41% of known relevant chunks are found in top-5 — the relevant content is in the index (otherwise recall would be 0); the problem is the retrieval system's ability to surface it, not a gap in the knowledge base. **Source:** Section 2: Data Preparation – Objective 6 & 7: Evaluate retrieval and advanced chunking — docs.databricks.com (search: "MLflow evaluate for RAG" and "Advanced chunking strategies RAG")

Question 17

Difficulty: Proficiency

A re-ranking stage is added after Vector Search retrieval in a RAG pipeline. Vector Search returns the top-50 candidate chunks; the reranker scores all 50 and passes the top-5 to the LLM. A senior engineer says: "If the correct chunk isn't in the top-50 from Vector Search, the reranker cannot help." Is this statement accurate, and what design implication does it have?

- **A)** The statement is inaccurate — rerankers can retrieve additional chunks from the Vector Search index beyond the initial top-50 candidates if they detect that the top-50 set does not contain a high-confidence match.
- **B)** The statement is accurate — the reranker only scores candidates already retrieved by Vector Search. If the relevant chunk is ranked 51st or lower by Vector Search, it is never seen by the reranker. The design implication is that the initial Vector Search `num_results` (retrieval pool size) must be large enough to include all likely relevant chunks before reranking.
- **C)** The statement is accurate — but this limitation is acceptable because Vector Search's approximate nearest neighbor algorithm guarantees that any chunk with cosine similarity above 0.7 to the query will always appear in the top-50 results.
- **D)** The statement is inaccurate — rerankers use a different embedding space than Vector Search, so a chunk ranked 51st by Vector Search may rank 1st by the reranker; both systems are queried independently and their results are merged.

Correct Answer: B **Explanation:** B is correct. The reranker is a second-stage filter, not an independent retrieval system. It receives only the candidates that Vector Search already returned (top-50 in this case). If the correct chunk is ranked 51st by Vector Search — even by the smallest cosine similarity margin — the reranker never sees it and cannot rescue it. The design implication is that the initial retrieval pool (`num_results`) must be generously sized to capture all potentially relevant chunks before the reranker applies its more precise scoring. A is wrong because rerankers in standard RAG architectures do not make additional retrieval calls to the Vector Search index — they operate purely on the already-retrieved candidate set. C is wrong because approximate nearest neighbor algorithms have no such guarantee — they trade some accuracy for speed, and chunks with borderline similarity may or may not appear in the top-50. D is wrong because the reranker and Vector Search use different scoring mechanisms (cross-encoder vs. bi-encoder), but they are not independent retrieval systems — the reranker receives only the Vector Search output; it does not query the index independently. **Source:** Section 2: Data Preparation – Objective 8: Explain the role of re-ranking in the information retrieval process — docs.databricks.com (search: "Databricks Vector Search reranking")

Question 18

Difficulty: Proficiency

A developer writes the following Spark code to prepare chunked data for Vector Search:

```
df.write.format("delta").mode("overwrite").saveAsTable("main.docs.chunks")
```

Then creates the Vector Search index immediately. Three days later, 200 new chunks are added to the source Delta table using `df_new.write.format("delta").mode("append").saveAsTable("main.docs.chunks")`. The developer triggers a Vector Search sync and expects the 200 new chunks to appear in the index. Instead, the sync fails with a "Change Data Feed not enabled" error. What went wrong?

- **A)** The `mode("overwrite")` used in the initial write destroyed and recreated the Delta table, which resets all table properties including `delta.enableChangeDataFeed = true` if it was previously set on the original table.
- **B)** The `mode("append")` used for the new chunks created a separate Delta table partition that is not covered by the CDF property set on the main table, requiring a separate CDF enable statement for the new partition.
- **C)** CDF was never enabled on the table before or after the initial write; the developer forgot to run `ALTER TABLE main.docs.chunks SET TBLPROPERTIES (delta.enableChangeDataFeed = true)` before creating the Vector Search index.
- **D)** The Vector Search index requires CDF to be enabled on the Unity Catalog schema (`main.docs`), not on the individual table; the developer incorrectly applied CDF at the table level instead of the schema level.

Correct Answer: C **Explanation:** C is correct. The most likely root cause is that CDF was never enabled on the table. The code shown (`write.format("delta")...saveAsTable()`) creates a Delta table but does not enable CDF. The Vector Search index was created without CDF being active, and when new data was appended and a sync was triggered, Vector Search detected the missing CDF property and threw the error. The fix is to run `ALTER TABLE main.docs.chunks SET TBLPROPERTIES (delta.enableChangeDataFeed = true)` after the initial table creation and before creating the index. A is wrong but is a relevant edge case: `mode("overwrite")` CAN reset table properties if it uses `overwriteSchema = true` or drops and recreates the table; however, in standard behavior `overwrite` mode replaces data files but preserves table metadata and properties. C is the most likely cause given the scenario. B is wrong because Delta Lake CDF is a table-level property — it applies to all partitions of the table uniformly; there is no per-partition CDF enablement. D is wrong because Unity Catalog does not have a schema-level CDF setting; CDF is configured at the individual Delta table level. **Source:** Section 2: Data Preparation – Objective 4: Define operations and sequence to write chunked text into Delta Lake — docs.databricks.com (search: "Enable Change Data Feed")

Question 19

Difficulty: Proficiency

An enterprise RAG pipeline processes 500,000 legal documents. After filtering, chunking, and embedding, the `Precision@5 = 0.88` but `Recall@5 = 0.31`. A principal engineer states: "High precision with low recall is the worst failure mode for a legal RAG application." Why is low recall especially dangerous in this legal context, and what architectural change best addresses it?

- **A)** Low recall means the chatbot frequently returns verbose answers with too much retrieved context (88% of retrieved chunks are relevant, creating information overload for the LLM), which is dangerous in legal contexts requiring concise citations.
- **B)** Low recall means the system retrieves 88% relevant chunks but misses 69% of all relevant legal information that exists in the knowledge base — in legal contexts, missing a relevant precedent, clause, or exception could lead to legally incorrect guidance; increasing `num_results` and adding a reranker improves recall without sacrificing precision.
- **C)** Low recall means 31% of all user queries fail to retrieve any results at all, which is dangerous because legal professionals receive empty answers for nearly 1 in 3 queries, requiring a fallback to manual document search.
- **D)** Low recall indicates that the Delta table holding the chunks has a low Change Data Feed transaction log retention period, causing 69% of indexed chunks to expire and become unavailable to Vector Search queries.

Correct Answer: B Explanation: B is correct. `Recall@5 = 0.31` means that for each query, only 31% of all truly relevant chunks in the knowledge base are being found in the top-5 results. In legal RAG, missing relevant information is catastrophic: an overlooked contract clause, missed legal precedent, or unstated exception could lead to legally incorrect advice with serious consequences. The fix is to increase the retrieval pool (`num_results`) so more candidates are retrieved initially, then apply a reranker to select the best 5 from a larger, higher-recall candidate set. A is wrong because high `Precision@5 = 0.88` means 88% of what IS retrieved is relevant — this is not information overload but high precision retrieval; the problem is what's being missed, not what's being retrieved. C is wrong because `Recall@k` does not measure the percentage of queries that return zero results; it measures coverage of known relevant documents within the top-k results for queries that do return results. D is wrong because `Recall@5` is a retrieval quality metric, not a Delta Lake storage metric; CDF log retention affects sync performance, not the retrieval recall of indexed content. **Source:** Section 2: Data Preparation – Objective 6: Use tools and metrics to evaluate retrieval performance — docs.databricks.com (search: "MLflow evaluate for RAG")

Question 20

Difficulty: Proficiency

A RAG pipeline uses the following two-stage retrieval: Vector Search retrieves top-100 chunks, then a cross-encoder reranker scores them and passes top-5 to the LLM. After deployment, latency is 4.2 seconds per query (SLA: 2 seconds). Profiling shows: Vector Search = 0.3s, reranker = 3.7s, LLM = 0.2s. The bottleneck is clear. What architectural change best reduces latency while maintaining retrieval quality?

- **A)** Replace the cross-encoder reranker with a second bi-encoder embedding model that scores all 100 chunks against the query using cosine similarity, which runs in parallel and is significantly faster than the cross-encoder.
- **B)** Reduce the Vector Search initial retrieval from top-100 to top-10, which gives the reranker only 10 chunks to score instead of 100, reducing reranker compute time by ~90% while accepting a small recall trade-off.
- **C)** Move the reranker to a dedicated Databricks Model Serving endpoint with GPU-backed autoscaling, so it processes all 100 chunks in parallel on GPU hardware rather than sequentially on CPU.
- **D)** Eliminate the reranker entirely and rely solely on Vector Search's cosine similarity ranking, reverting to the original retrieval approach and accepting the quality reduction as the only option within the latency constraint.

Correct Answer: B Explanation: B is correct. The reranker's 3.7s latency scales with the number of chunks it must score — cross-encoders process each query-chunk pair individually. Reducing the candidate pool from 100 to 10 reduces the reranker's work by ~90%, bringing estimated reranker time to ~0.37s and total latency to approximately 0.87s — well within the 2-second SLA. The trade-off is a modest decrease in recall (fewer candidates for the reranker to choose from), which the team must validate against quality requirements. A is wrong because a second bi-encoder (the same technology as Vector Search) simply re-ranks using cosine similarity — it does not achieve the precision improvement of a cross-encoder. Using a bi-encoder for reranking defeats the purpose of the reranking stage. C is wrong because while GPU-backed serving would improve throughput for many concurrent users, scoring 100 query-document pairs sequentially on GPU reduces latency somewhat but not by the ~85% needed to meet the 2-second SLA given the current architecture. D is wrong because eliminating the reranker should be the last resort; reducing the candidate pool (option B) achieves the latency target while preserving much of the quality benefit of reranking. **Source:** Section 2: Data Preparation – Objective 8: Explain the role of re-ranking in the information retrieval process — docs.databricks.com (search: "Databricks Vector Search reranking" and "Foundation Model APIs cross-encoders")

Question 21

Difficulty: Beginner

A developer needs to split a 50-page PDF user manual into chunks for a RAG pipeline. The manual is written in continuous plain prose with no consistent heading structure. Which chunking method is most reliable given this document structure?

- **A)** Document-aware chunking using Markdown header boundaries — the splitter detects `#`, `##`, and `###` headers and uses them as split points to keep related sections together in each chunk.
- **B)** Fixed-size chunking with an appropriate token limit and small overlap — it splits the text at a consistent token count with overlap to avoid losing context at boundaries, regardless of document structure.
- **C)** Semantic chunking using a topic model — the chunker runs an LDA topic model on the full PDF and groups sentences by topic, placing all sentences with the same dominant topic into a single chunk.
- **D)** Table-based chunking — the chunker detects table boundaries in the PDF and uses each table cell as a single chunk, which is appropriate for prose documents that lack explicit section headers.

Correct Answer: B **Explanation:** B is correct. For a plain prose document with no consistent structural markers (no Markdown headers, no HTML tags, no section dividers), document-aware or header-based chunking cannot find split points. Fixed-size chunking with overlap is the reliable fallback — it splits at a consistent token count and uses overlap to prevent losing context at chunk boundaries. A is wrong because document-aware Markdown header chunking requires the document to actually have Markdown headers (`#` , `##`); a PDF with plain prose has no such headers and the splitter would find no split points or produce a single giant chunk. C is wrong because running an LDA topic model as a chunking strategy is extremely computationally expensive, slow, and non-standard for production RAG pipelines; topic modeling at inference time is not a supported chunking strategy on Databricks. D is wrong because table-based chunking is designed for documents with tabular data, not continuous prose; applying it to prose would produce incorrect and nonsensical splits. **Source:** Section 2: Data Preparation – Objective 1: Apply a chunking strategy for a given document structure and model constraints — docs.databricks.com (search: "Data preparation for RAG applications")

Question 22

Difficulty: Beginner

A developer is preparing a Delta table to serve as the source for a Databricks Vector Search index. They realize the table has no primary key column. Why is a unique `id` column required?

- **A)** The Vector Search index uses the `id` column as the vector embedding storage key, mapping each unique ID to its corresponding high-dimensional vector in the FAISS index file.
- **B)** The `id` column is required so that Vector Search can uniquely identify each chunk during delta sync operations — specifically to detect which chunks have been updated or deleted and apply those changes incrementally.
- **C)** Unity Catalog enforces that all Delta tables used by AI services must have a primary key column for compliance auditing, so that the data lineage of each chunk can be traced back to its original document.
- **D)** The `id` column is used by the LLM at inference time to cite the exact source document and chunk position in its response, enabling automatic footnote generation in the chatbot's answers.

Correct Answer: B **Explanation:** B is correct. Databricks Vector Search uses the `id` column to uniquely identify each row (chunk) in the source Delta table. During incremental sync operations, the Vector Search pipeline uses Change Data Feed records alongside the `id` to determine which specific rows have been inserted, updated, or deleted — and applies those changes to the index precisely. Without a unique `id`, the index cannot correctly perform row-level updates or deletions. A is wrong because the `id` column is a logical identifier for the chunk in the Delta table, not the key used to store the embedding vector in the underlying vector storage; embeddings are stored internally by the Vector Search service. C is wrong because Unity Catalog does not require a primary key column for AI service compliance; the `id` requirement is a Vector Search operational requirement, not a governance policy. D is wrong because the LLM does not use the `id` column from the Delta table at inference time; citation is a separate application-level feature built on top of the retrieved chunk metadata, not an automatic behavior driven by the `id` column. **Source:** Section 2: Data Preparation – Objective 4: Define operations and sequence to write chunked text into Delta Lake — [docs.databricks.com](https://docs.databricks.com/search-and-indexing/vector-search.html) (search: "Create a Mosaic AI Vector Search index")

Question 23

Difficulty: Beginner

A developer uses `beautifulsoup4` to extract text from HTML pages. They run `soup.get_text()` and get back a string, but it contains many unwanted newlines, tab characters, and whitespace runs from the original HTML formatting. What should they do before writing this text to a Delta table for chunking?

- **A)** Re-parse the HTML with `pytesseract` OCR after rendering the page to an image, which produces cleaner text by re-reading the visual characters rather than the raw HTML source text.
- **B)** Apply string cleaning operations — strip leading/trailing whitespace, collapse multiple consecutive whitespace characters, and remove newlines from within sentences — to normalize the extracted text.
- **C)** Write the raw uncleaned text to the Delta table and rely on the embedding model to normalize whitespace during the vectorization process, since embedding models ignore whitespace tokens.
- **D)** Convert the HTML to PDF first using `pdftkit`, then use `PyPDF2` to extract clean text, which strips HTML formatting artifacts and produces uniform paragraph text automatically.

Correct Answer: B **Explanation:** B is correct. After extracting raw text from HTML using `soup.get_text()`, the output typically contains formatting artifacts from the original HTML structure — excessive newlines (from `<br>`, `<p>` tags), tabs, and whitespace runs. Standard text cleaning (using Python's `str.strip()`, `re.sub(r'\s+', ' ', text)`, and similar operations) normalizes the text before it is chunked and embedded, ensuring cleaner, higher-quality chunks. A is wrong because converting a text-based HTML page to an image and running OCR is a wasteful, lossy approach; OCR introduces errors and is designed for image-based documents where digital text is unavailable. C is wrong because embedding models do NOT normalize whitespace — they tokenize the input as-is, and excessive whitespace tokens reduce the effective content density of the embedding. D is wrong because HTML → PDF → text extraction is a multi-step conversion that adds complexity and potential content loss; direct HTML text cleaning via BeautifulSoup is simpler, faster, and more reliable. **Source:** Section 2: Data Preparation – Objective 2: Filter extraneous content in source documents — docs.databricks.com (search: "Generative AI data preparation")

Question 24

Difficulty: Beginner

A team has a collection of `.docx` Word documents that they want to parse for a RAG pipeline. Which Python library is most appropriate for extracting text content from Word documents on Databricks?

- **A)** `beautifulsoup4` — because Word documents are internally structured as ZIP archives containing XML files, and BeautifulSoup can parse XML to extract text from the document body elements.
- **B)** `pytesseract` — because Word documents are often printed and scanned, making OCR the standard text extraction approach for this document format in enterprise pipelines.
- **C)** `unstructured` — because it natively supports `.docx` Word document parsing, extracting text with structural awareness (distinguishing headings from body text) without requiring manual XML parsing.
- **D)** `PyPDF2` — because Word documents and PDF files use the same underlying binary format, and PyPDF2's universal document reader handles both `.pdf` and `.docx` file extensions.

Correct Answer: C **Explanation:** C is correct. The `unstructured` library is the recommended multi-format document parsing tool for RAG pipelines. It natively reads `.docx` files using the `python-docx` backend and returns structured elements (Title, NarrativeText, Table, etc.), making it easy to filter and process different content types from Word documents. A is wrong because while Word's `.docx` format is internally an XML structure, using `beautifulsoup4` for XML parsing requires significant manual handling of namespace prefixes and schema knowledge; `unstructured` abstracts this complexity. B is wrong because `.docx` files contain embedded digital text — OCR is only needed for scanned image files where no digital text layer exists. Using `pytesseract` on a standard Word document would be unnecessarily complex and lossy. D is wrong because Word and PDF are completely different binary formats with different internal structures; `PyPDF2` can only read PDF files and has no capability to parse `.docx` documents. **Source:** Section 2: Data Preparation – Objective 3: Choose the appropriate Python package — docs.databricks.com (search: "RAG reference architecture data preparation")

Question 25

Difficulty: Beginner

A developer is curating documents for a customer support RAG chatbot. They have access to: (A) official product FAQs updated monthly, (B) internal Slack message threads from the support team, and (C) competitor product reviews from public websites. Which documents should be excluded, and why?

- **A)** Exclude source A (official FAQs) because monthly updates mean the information is stale for 30 days; real-time Slack threads (source B) provide more current information for customer queries.
- **B)** Exclude sources B and C — Slack threads contain informal, unverified, and potentially incorrect troubleshooting advice, and competitor reviews introduce off-topic and potentially misleading information into the knowledge base.
- **C)** Include all three sources to maximize knowledge breadth; the LLM will automatically disregard low-quality content from Slack threads and competitor reviews when generating answers for customer queries.
- **D)** Exclude source A (official FAQs) and include only sources B and C because the chatbot needs to understand real-world customer language patterns, which Slack threads and public reviews provide more authentically.

Correct Answer: B **Explanation:** B is correct. Slack threads (B) contain informal, unverified advice — support agents may share workarounds, incorrect steps, or personal opinions that are not official guidance. Including these pollutes the knowledge base with potentially incorrect information. Competitor product reviews (C) are entirely off-topic for a customer support chatbot — they would cause the retriever to surface competitor product information in response to customer queries, which is harmful and potentially embarrassing. Only the official FAQs (A) provide verified, authoritative, on-topic content. A is wrong because monthly-updated official FAQs are authoritative — even if 30 days old, they represent the current official guidance. Slack threads are NOT more reliable simply because they are more recent. C is wrong because LLMs cannot reliably "ignore" retrieved context that they are given; if the retriever returns incorrect Slack advice, the LLM is likely to incorporate it into the answer. D is wrong because understanding customer language patterns is a training-time concern for the LLM, not a knowledge base curation decision; the knowledge base should contain only authoritative, accurate information. **Source:** Section 2: Data Preparation – Objective 5: Identify needed source documents — docs.databricks.com (search: "Generative AI data preparation best practices")

Question 26

Difficulty: Intermediate

A developer uses recursive character text splitting on a 200-page technical manual. They set `chunk_size=400` tokens and `chunk_overlap=40` tokens. They observe that some chunks contain only 15–20 tokens (mostly section titles that appear alone after a split). What is the impact of these tiny chunks on the RAG pipeline, and what is the fix?

- **A)** Tiny chunks improve retrieval precision because the embedding model can produce highly focused embeddings for short, specific text; the developer should keep them and reduce `chunk_size` further to create more fine-grained chunks.
- **B)** Tiny chunks create poor embeddings that are underrepresented in the vector space — their embedding captures only a heading with minimal semantic content, meaning they will consume a top-k retrieval slot while providing almost no useful context to the LLM. The fix is to set a `min_chunk_size` threshold to merge or discard chunks below a minimum token count.
- **C)** Tiny chunks only affect the storage cost of the Delta table since each row occupies a fixed storage overhead regardless of text length; they have no meaningful impact on retrieval quality or LLM answer generation.
- **D)** Tiny chunks cause the Vector Search index to fail during sync because the embedding model raises an error when it receives fewer than 50 tokens as input, rejecting the chunk and preventing it from being added to the index.

Correct Answer: B **Explanation:** B is correct. A chunk containing only a section heading like "Chapter 4: Configuration" has minimal semantic content — its embedding is essentially a representation of a header phrase, not of any knowledge. When retrieved, it provides almost no useful context for the LLM to generate an answer, yet it occupies one of the precious top-k retrieval slots. The fix is to apply a `min_chunk_size` filter (e.g., 50 tokens) that either merges tiny chunks with the next chunk or discards them entirely. A is wrong because there is a meaningful lower bound below which chunk size hurts quality rather than helping precision — a 15-token heading phrase does not benefit from a focused embedding; it simply lacks content. C is wrong because tiny chunks do meaningfully impact retrieval quality by occupying retrieval slots with low-information-content embeddings, degrading the quality of context passed to the LLM. D is wrong because embedding models accept input strings of any length down to a single token; they do not raise errors for short inputs, though the quality of very short-text embeddings is poor. **Source:** Section 2: Data Preparation – Objective 1: Apply a chunking strategy — docs.databricks.com (search: "Data preparation for RAG applications")

Question 27

Difficulty: Intermediate

A team uses `mlflow.genai.evaluate()` to assess retrieval quality and gets `Precision@5 = 0.76` and `Recall@5 = 0.52`. The product manager asks which metric is more important for a customer-facing medical symptom checker chatbot. What is the correct reasoning?

- **A)** Precision@5 is more important because the medical chatbot must never return irrelevant information — a hallucinated medical fact from an off-topic chunk could cause the user to make a dangerous health decision.
- **B)** Recall@5 is more important because missing a relevant medical document (e.g., a critical drug interaction warning) is more dangerous than retrieving a slightly off-topic chunk; the chatbot must ensure it surfaces all relevant medical information.
- **C)** Both metrics are equally important and must both reach 1.0 before a medical chatbot can be deployed; partial retrieval quality is not acceptable in any medical application context.
- **D)** Neither metric is relevant for a medical chatbot; the correct metric is `AnswerCorrectness`, which measures whether the final LLM response is factually correct, not whether individual retrieved chunks are relevant.

Correct Answer: B **Explanation:** B is correct. In a medical context, missing relevant information (low recall) is the more dangerous failure mode. A missed drug interaction warning or contraindication could lead a user to make an unsafe decision. Retrieving a slightly off-topic chunk (lower precision) is less dangerous because the LLM is less likely to synthesize irrelevant text into a harmful answer. This is a classic recall vs. precision prioritization decision based on the asymmetry of error consequences. A is wrong because while precision matters, the medical context prioritizes not missing critical information over not retrieving irrelevant information — the asymmetry of harm favors recall. C is wrong because perfect scores (1.0) are not achievable in real-world RAG systems due to query-document semantic gaps and retrieval model limitations; setting 1.0 as a deployment criterion is impractical. D is wrong because retrieval metrics (Precision@k, Recall@k) and answer metrics (AnswerCorrectness) both matter and measure different things; dismissing retrieval metrics is incorrect — poor retrieval directly causes poor answer quality. **Source:** Section 2: Data Preparation – Objective 6: Use tools and metrics to evaluate retrieval performance — docs.databricks.com (search: "MLflow evaluate for RAG")

Question 28

Difficulty: Intermediate

A developer wants to ensure that when the source Delta table `main.docs.policy_chunks` is updated with new policy documents, the changes propagate to the Vector Search index automatically without manual intervention. They configure the Vector Search index with `pipeline_type="CONTINUOUS"`. What prerequisite must be met on the source Delta table?

- **A)** The source Delta table must be partitioned by the `document_date` column so that the CONTINUOUS sync pipeline can efficiently identify which partitions contain new data during each micro-batch scan.
- **B)** The source Delta table must have Change Data Feed enabled (`delta.enableChangeDataFeed = true`) so the CONTINUOUS sync pipeline can read the incremental change records and propagate only the new, updated, or deleted rows.
- **C)** The source Delta table must be registered in the Hive metastore (not Unity Catalog) because CONTINUOUS sync mode only supports Hive metastore tables, not Unity Catalog-managed tables with governance policies.
- **D)** The source Delta table must be stored on Azure Data Lake Storage Gen2 or AWS S3, because CONTINUOUS sync mode requires a cloud object storage backend; local cluster storage (DBFS root) is not supported for real-time sync pipelines.

Correct Answer: B **Explanation:** B is correct. Databricks Vector Search CONTINUOUS sync uses Change Data Feed to detect and propagate changes from the source Delta table in near-real-time. Without CDF enabled on the source table, the sync pipeline has no mechanism to detect incremental changes and will fail. CDF must be enabled before the Vector Search index is created (using `ALTER TABLE ... SET TBLPROPERTIES (delta.enableChangeDataFeed = true)`). A is wrong because partitioning by date is a query optimization for analytics, not a prerequisite for Vector Search sync; CONTINUOUS mode reads CDF records regardless of partition structure. C is wrong because Databricks Vector Search fully supports Unity Catalog-managed Delta tables — Unity Catalog is in fact the recommended governance layer for Vector Search source tables. D is wrong because CONTINUOUS sync requirements relate to CDF (a Delta Lake feature) and Unity Catalog governance, not to specific cloud storage backends; DBFS root tables can also be used, though Unity Catalog-managed tables are preferred. **Source:** Section 2: Data Preparation – Objective 4: Define operations and sequence to write chunked text into Delta Lake — docs.databricks.com (search: "Create a Mosaic AI Vector Search index")

Question 29

Difficulty: Intermediate

A developer is building a RAG pipeline for a company that sells both software products and hardware products. The knowledge base contains 50,000 chunks covering both product lines. Users sometimes ask software questions but retrieve hardware documentation and vice versa. What data preparation change best addresses this cross-contamination problem?

- **A)** Increase the Vector Search index `num_results` to 20 so the correct chunks from the right product line are more likely to appear somewhere in the retrieved set before passing them to the LLM.
- **B)** Add a `product_line` metadata column (`software` or `hardware`) to each chunk in the source Delta table, and apply metadata filtering at query time so each query only searches within the relevant product line's chunks.
- **C)** Train a custom fine-tuned embedding model on combined software and hardware documentation so the embedding space naturally separates the two product domains and reduces cross-domain retrieval confusion.
- **D)** Create two separate Databricks workspaces — one for the software knowledge base and one for the hardware knowledge base — routing user queries to the appropriate workspace based on keyword matching.

Correct Answer: B **Explanation:** B is correct. Adding a `product_line` metadata filter is the most targeted and operationally efficient fix. Databricks Vector Search supports metadata filtering — you can add a column (e.g., `product_line: string`) to the source Delta table and pass a filter expression at query time (e.g., `filters_json='{\"product_line\": \"software\"}'`) to restrict the search to only relevant chunks. This eliminates cross-contamination at the retrieval layer without requiring model retraining or architectural changes. A is wrong because increasing `num_results` retrieves more chunks but does not filter by product line — you'd get more chunks from both product lines, making the cross-contamination problem potentially worse as more off-topic chunks are passed to the LLM. C is wrong because fine-tuning a custom embedding model is an expensive, time-consuming process that requires labeled training data; metadata filtering achieves the same result with zero training cost. D is wrong because splitting into two workspaces is extreme engineering overhead — separate Unity Catalog schemas or separate Vector Search indexes within one workspace achieve the same isolation far more efficiently. **Source:** Section 2: Data Preparation – Objective 5: Identify needed source documents — docs.databricks.com (search: "Mosaic AI Vector Search index" and "Generative AI data preparation best practices")

Question 30

Difficulty: Intermediate

A developer instruments their RAG retrieval function with `@mlflow.trace(span_type="RETRIEVER")`. When they inspect the captured trace in the MLflow UI, what information appears in this span?

- **A)** The span shows the LLM's generated response along with the confidence scores that the LLM internally assigned to each retrieved chunk when deciding how much weight to give each document.
- **B)** The span shows the query vector (the numerical embedding array) that was sent to the Vector Search index and the raw cosine similarity scores of the top-k matching embeddings.
- **C)** The span shows the input query string, the retrieved document chunks (text content and metadata), and timing information — allowing MLflow evaluators to assess what the retriever received and returned.
- **D)** The span shows the Delta table transaction log entries that were read during the retrieval, including the CDF records that indicate which rows were recently updated in the source table.

Correct Answer: C **Explanation:** C is correct. The `RETRIEVER` span type in MLflow Tracing captures the inputs and outputs of the retrieval step as structured metadata in the trace tree. Specifically, it records the input (the user query string), the outputs (the retrieved document chunks, including their text content and metadata such as `doc_id`, `source`, similarity score), and timing information. MLflow's `RetrievalRelevance` scorer then reads these captured inputs/outputs from the span to perform LLM-judge-based relevance evaluation. A is wrong because LLMs do not expose internal confidence scores per retrieved document — the LLM processes the assembled prompt without providing per-chunk weighting metadata back to the application. B is wrong because the `RETRIEVER` span captures the text-level inputs and outputs for evaluation purposes — it does not expose the raw numerical embedding vector or internal cosine similarity computations from the Vector Search service. D is wrong because the `RETRIEVER` span captures the RAG pipeline's retrieval step inputs/outputs, not the underlying Delta table transaction log; CDF records are an internal sync mechanism, not exposed in the retrieval trace. **Source:** Section 2: Data Preparation – Objective 6: Use tools and metrics to evaluate retrieval performance — [docs.databricks.com](https://docs.databricks.com/search/mlflow-tracing) (search: "MLflow Tracing")

Question 31

Difficulty: Advanced

A developer applies document-aware chunking to a Markdown knowledge base. Articles are split on `##` headers. One article's section titled "### Troubleshooting" contains 4,200 tokens — far exceeding the 512-token embedding model limit. What is the correct approach to handle this oversized section?

- **A)** Skip the "Troubleshooting" section entirely and exclude it from the knowledge base because sections exceeding the embedding model limit cannot be ingested without data loss.
- **B)** Apply a two-level chunking strategy: use header-based splitting at the `##` level first, then apply recursive character splitting on any resulting chunk that exceeds 512 tokens, preserving header-level structure where possible.
- **C)** Increase the embedding model's token limit by setting a `max_tokens=4200` parameter in the embedding model API call, which instructs the model to expand its context window for this specific oversized section.
- **D)** Replace the document-aware chunker with a single global fixed-size chunker set to 4,200 tokens across the entire knowledge base so that the largest section fits within one chunk without being split.

Correct Answer: B **Explanation:** B is correct. A two-level approach handles this gracefully: first split on `##` headers to preserve section-level structure (which is the goal of document-aware chunking), then apply recursive character splitting to any section that exceeds the 512-token limit to bring it within bounds. This preserves as much semantic structure as possible while respecting the hard model constraint. A is wrong because skipping entire sections means losing potentially critical knowledge — the troubleshooting section is likely highly relevant to user queries and should not be excluded. C is wrong because embedding model context windows are fixed by the model architecture; there is no `max_tokens` parameter that expands the model's capacity at inference time. D is wrong because setting a 4,200-token chunk size breaks all other sections in the knowledge base — short sections would become tiny chunks, and most sections would vastly exceed the 512-token embedding limit, degrading the entire pipeline for the sake of one outlier section. **Source:** Section 2: Data Preparation – Objective 1 & 7: Chunking strategy and advanced chunking — docs.databricks.com (search: "Advanced chunking strategies RAG")

Question 32

Difficulty: Advanced

A RAG evaluation shows `Precision@5 = 0.94` but the LLM is still generating answers with hallucinations and missing important details. The retrieved chunks are highly relevant but engineers discover the LLM context window only receives ~400 tokens per query (limited by the 5 small child chunks of ~80 tokens each in a Parent-Child setup). What is the architectural fix?

- **A)** Reduce the number of retrieved chunks from 5 to 2 to concentrate the LLM's attention on the most relevant content, which reduces hallucinations by minimizing the amount of context the LLM must process.
- **B)** Switch from Parent-Child chunking to pure small-chunk retrieval, removing the parent chunk fetching step to reduce context size and make the LLM rely more on its parametric knowledge for complete answers.
- **C)** The child chunks are correctly retrieved (high precision), but the LLM receives only child-level context (80 tokens each). The fix is to implement the Parent-Child fetch correctly — retrieve parent chunks (~500–1,000 tokens) corresponding to the matched children, providing richer context to the LLM.
- **D)** Add more Vector Search metadata filters to narrow the retrieval set further, which reduces the number of chunks from 5 to 1–2 and forces the LLM to focus exclusively on the single most relevant passage.

Correct Answer: C **Explanation:** C is correct. This is exactly the problem Parent-Child chunking is designed to solve — and the description reveals it's not being implemented correctly. High Precision@5 confirms the right child chunks are being found. But if only the small child chunks (80 tokens) are passed to the LLM, the LLM lacks sufficient surrounding context for complete answers. The correct implementation fetches the parent chunk (the larger parent section, e.g., 500–1,000 tokens) corresponding to each matched child chunk and sends those parent chunks to the LLM instead. This provides rich context while maintaining precise retrieval. A is wrong because reducing from 5 to 2 chunks reduces the amount of context even further, exacerbating the problem. B is wrong because removing the parent chunk step means losing the architectural benefit entirely — without parent chunks, the LLM still only gets 80-token child chunks, which is the exact problem. D is wrong because further narrowing to 1–2 chunks reduces context coverage and increases the risk of missing important details rather than fixing the hallucination caused by insufficient context. **Source:** Section 2: Data Preparation – Objective 7: Design retrieval systems using advanced chunking strategies — [docs.databricks.com](https://docs.databricks.com/search/advanced-chunking-strategies-rag) (search: "Advanced chunking strategies RAG")

Question 33

Difficulty: Advanced

A developer runs `mlflow.genai.evaluate()` on a RAG pipeline and gets `RetrievalRelevance = 0.89`. They present this to a stakeholder who asks: "Does this mean 89% of our retrieved chunks are factually correct?" How should the developer respond?

- **A)** Yes — `RetrievalRelevance = 0.89` means that the LLM judge confirmed 89% of the retrieved chunks contain factually accurate information verified against an external truth source.
- **B)** No — `RetrievalRelevance` measures whether retrieved chunks are relevant to the user's query (topically related), not whether their content is factually correct. A chunk can be highly relevant (about the right topic) but contain outdated or incorrect facts.
- **C)** Yes — `RetrievalRelevance` uses an LLM judge that cross-references each retrieved chunk against a verified ground-truth database, so a score of 0.89 confirms 89% of chunks are both relevant and factually verified.
- **D)** No — `RetrievalRelevance = 0.89` means that 89% of all the chunks in the Vector Search index (not just retrieved chunks) are relevant to the total set of queries in the evaluation dataset, measuring overall index coverage.

Correct Answer: B **Explanation:** B is correct. `RetrievalRelevance` is a topical relevance metric — the LLM judge assesses whether each retrieved chunk is about the right topic relative to the user's query. It does not verify factual accuracy. A chunk about "password reset procedures" is highly relevant to a "how do I reset my password?" query even if the reset steps described are outdated and incorrect. Factual correctness is measured by a different set of metrics (e.g., `AnswerCorrectness` with ground truth, or `Groundedness` for answer-context consistency). A is wrong because `RetrievalRelevance` does not perform factual verification — it only assesses topical relevance using an LLM judge that has no access to an external ground-truth database. C is wrong for the same reason as A — no ground-truth database is consulted during `RetrievalRelevance` scoring. D is wrong because `RetrievalRelevance` measures the quality of retrieved chunks for specific queries in the evaluation set, not the overall relevance of all chunks in the entire index. **Source:** Section 2: Data Preparation – Objective 6: Use tools and metrics to evaluate retrieval performance — [docs.databricks.com](https://docs.databricks.com/search/mlflow-evaluate-for-rag) (search: "MLflow evaluate for RAG")

Question 34

Difficulty: Advanced

A developer filters extraneous content from HTML documents using BeautifulSoup. Their cleaning function removes all `<nav>`, `<footer>`, `<header>`, and `<aside>` tags. During evaluation, users report the chatbot cannot answer questions about table data from the source HTML pages. Investigation reveals the HTML tables are being stripped. What went wrong?

- **A)** BeautifulSoup's `get_text()` method automatically strips all HTML table content (`<table>`, `<tr>`, `<td>` tags) regardless of the developer's filter configuration, converting tables to empty strings.
- **B)** The developer's filter is too broad — their code likely removes all tags rather than only the specified structural tags, stripping `<table>` elements along with the navigation and footer elements they intended to remove.
- **C)** HTML tables require a separate `pytesseract` OCR pass because BeautifulSoup cannot parse `<table>` tags in standard HTML; tables are treated as binary image elements by the HTML parser.
- **D)** The Vector Search embedding model does not support tabular data formats — when a chunk contains HTML table content, the embedding computation fails silently and the chunk is not indexed.

Correct Answer: B **Explanation:** B is correct. The most likely error is over-aggressive filtering. A common mistake is using `soup.get_text()` directly after removing specific tags, or applying a filter like `for tag in soup.find_all(True): tag.decompose()` that removes ALL tags (including `<table>`, `<tr>`, `<td>`) instead of only the intended structural navigation elements. The developer should specifically target only `<nav>`, `<footer>`, `<header>`, `<aside>` tags for removal and explicitly preserve `<table>` content. Using `unstructured` instead of raw BeautifulSoup would handle table extraction as a separate element type automatically. A is wrong because BeautifulSoup's `get_text()` does extract text from table cells — it does not strip table content automatically; the issue is in the developer's custom filtering code. C is wrong because BeautifulSoup is a full HTML parser that handles `<table>` tags correctly; tables are not binary image elements and do not require OCR. D is wrong because Vector Search embedding models accept any text string, including text extracted from tables; there is no special failure mode for tabular content. **Source:** Section 2: Data Preparation – Objective 2: Filter extraneous content in source documents — [docs.databricks.com](https://docs.databricks.com/search/generative-ai/data-preparation.html) (search: "Generative AI data preparation")

Question 35

Difficulty: Advanced

A data engineer is preparing a RAG pipeline for a healthcare company. Source documents include: clinical trial reports (PDF), drug interaction databases (CSV exported to plain text), and patient intake forms (scanned paper forms as image PDFs). The engineer must select a different extraction tool for each category. What is the correct tool mapping?

- **A)** Clinical trial PDFs → `PyPDF2` ; drug interaction CSV text → `unstructured` ; scanned patient forms → `pytesseract` . This mapping applies the appropriate tool to each format's specific characteristics.
- **B)** Clinical trial PDFs → `beautifulsoup4` ; drug interaction CSV text → `PyPDF2` ; scanned patient forms → `unstructured` . BeautifulSoup handles all PDF types and PyPDF2 is for CSV formats.
- **C)** Clinical trial PDFs → `pytesseract` ; drug interaction CSV text → `pytesseract` ; scanned patient forms → `pytesseract` . Pytesseract is the universal extraction tool for all healthcare document formats.
- **D)** Clinical trial PDFs → `unstructured` ; drug interaction CSV text → Python `csv` module or pandas `read_csv()` ; scanned patient forms → `pytesseract` . Each tool is matched to the format's specific extraction requirement.

Correct Answer: D **Explanation:** D is correct. The correct mapping uses the right tool for each document type: `unstructured` handles complex PDF clinical reports with mixed content (text, tables, headers); the Python `csv` module or pandas handles structured CSV plain-text data without needing an NLP library; `pytesseract` performs OCR on scanned image PDFs (patient intake forms) where no digital text layer exists. A is wrong because while `PyPDF2` can handle standard text-based PDFs, `unstructured` is superior for complex clinical PDFs with mixed content; using `unstructured` for CSV text adds unnecessary overhead when native CSV parsers are available. B is wrong because `beautifulsoup4` is an HTML parser that cannot parse PDF files; `PyPDF2` cannot parse CSV files. C is wrong because `pytesseract` is only appropriate for image/scanned documents; using it universally for digital PDFs and CSV text introduces unnecessary OCR processing and potential accuracy loss. **Source:** Section 2: Data Preparation – Objective 3: Choose the appropriate Python package — docs.databricks.com (search: "RAG reference architecture data preparation")

Question 36

Difficulty: Proficiency

A production RAG pipeline serves 50,000 queries per day. The Vector Search index has 2 million chunks. A data scientist runs a retrieval evaluation over a 500-query benchmark and reports $nDCG@10 = 0.61$. The engineering team lead asks: "Is this score good enough to ship, and what does $nDCG@10$ specifically measure that $Precision@10$ and $Recall@10$ do not?" Provide the technically complete answer.

- **A)** $nDCG@10 = 0.61$ is always sufficient for production deployment; $nDCG@10$ measures the same thing as $Precision@10$ but averages it across queries, providing a more statistically stable metric with no additional ranking information.
- **B)** Whether 0.61 is sufficient depends on the use case baseline; $nDCG@10$ measures position-weighted relevance — it rewards ranking highly relevant chunks first and penalizes placing them lower — capturing rank quality that neither $Precision@10$ (equal weight per rank) nor $Recall@10$ (coverage only) measures.
- **C)** $nDCG@10 = 0.61$ is always insufficient for production; $nDCG$ measures the fraction of queries where the first result is relevant, and 0.61 means 39% of queries have an irrelevant first result, which is unacceptable for production.
- **D)** $nDCG@10$ measures the same thing as $Recall@10$ but normalized between 0 and 1; a score of 0.61 means 61% of all relevant chunks in the index appear somewhere in the top-10 results across all evaluated queries.

Correct Answer: B **Explanation:** B is correct. $nDCG$ (Normalized Discounted Cumulative Gain) at k measures position-weighted relevance: a highly relevant chunk at rank 1 contributes much more to the score than the same relevant chunk at rank 10. It also handles graded relevance (a chunk can be more or less relevant, not just binary). Neither $Precision@10$ (fraction of top-10 that are relevant — equal weight per position) nor $Recall@10$ (fraction of all relevant chunks found in top-10) captures the ranking quality — they don't distinguish between finding the best chunk at rank 1 vs. rank 10. Whether 0.61 is "good enough" depends on the use case, the baseline of the previous system, and the business requirements — there is no universal threshold. A is wrong because $nDCG$ is NOT the same as averaged $Precision@k$ — $nDCG$ includes position discounting and handles graded relevance, providing additional ranking quality information. C is wrong because $nDCG$ is not the fraction of queries with a relevant first result — that is $Precision@1$ or a variation of MRR. D is wrong because that describes a version of normalized Recall, not $nDCG$; $nDCG$ measures position-weighted cumulative gain, not binary coverage of relevant chunks. **Source:** Section 2: Data Preparation – Objective 6: Use tools and metrics to evaluate retrieval performance — docs.databricks.com (search: "MLflow evaluate for RAG")

Question 37

Difficulty: Proficiency

A team is designing a RAG pipeline for a company's 10-year archive of engineering change orders (ECOs), each a 20–80 page PDF with structured sections (Summary, Affected Parts, Test Results, Approvals). They must choose between three chunking strategies: (A) fixed-size 512-token chunks, (B) section-aware chunking by ECO section headers, and (C) Parent-Child chunking with section headers as parent boundaries and 150-token sub-sections as children. What is the complete reasoning for selecting strategy C?

- **A)** Strategy C is the most complex and therefore always the best choice for production RAG pipelines — complexity in the chunking layer inversely correlates with hallucination rates in the LLM output.
- **B)** Strategy C is best because it combines section-aware structure preservation (parent boundaries align with ECO sections) with precise child-level embeddings for accurate retrieval, and full parent section delivery to the LLM for rich context — addressing all three requirements: structure, precision, and context.
- **C)** Strategy C should be avoided because the section headers (parent boundaries) in ECOs vary across 10 years of documents, making it impossible to reliably detect consistent header patterns for parent chunking.
- **D)** Strategy C is the best choice only if the ECOs are stored as Markdown files; for PDFs, parent-child chunking is not supported by any Databricks-compatible parsing library.

Correct Answer: B **Explanation:** B is correct. For structured, long-form technical documents like ECOs, Parent-Child chunking optimally addresses the three key requirements: (1) **Structure preservation** — parent boundaries align with ECO sections (Summary, Affected Parts, etc.), keeping logically related content together at the parent level. (2) **Retrieval precision** — small 150-token child chunks produce focused, precise embeddings that match specific query terms (e.g., "test results for part #XYZ") without the semantic diffusion of large chunks. (3) **Context richness** — the LLM receives the full parent section (potentially 500–2,000 tokens) containing the matched child, providing sufficient context for complete, well-grounded answers. A is wrong because complexity alone is never a selection criterion; the choice should be based on document structure, query patterns, and performance requirements. C is wrong because even if header patterns vary across 10 years, the `unstructured` library and similar tools can detect section boundaries from formatting cues (font size, capitalization, whitespace), and variable headers can be normalized during pre-processing. D is wrong because Parent-Child chunking is a logical strategy applied to any text, regardless of source format; `unstructured` parses PDFs into sections that can serve as parent boundaries. **Source:** Section 2: Data Preparation – Objective 7: Design retrieval systems using advanced chunking strategies — docs.databricks.com (search: "Advanced chunking strategies RAG")

Question 38

Difficulty: Proficiency

A Vector Search reranking stage uses a cross-encoder model hosted on a Databricks Model Serving endpoint. The initial Vector Search returns 50 candidates; the reranker scores and returns top-5. A developer observes that for very short, ambiguous queries like "error fix", the reranker consistently places a generic troubleshooting document first, while for detailed queries like "SSL certificate validation error in Python 3.11 requests library", the reranker performs excellently. Why does query length/specificity affect reranker performance, and what can be done to improve short-query performance?

- **A)** Short queries cause the cross-encoder model to time out on the Databricks Model Serving endpoint because the model requires a minimum input length to warm up its attention mechanism; padding short queries to 50 tokens fixes the timeout.
- **B)** Cross-encoders score query-document relevance jointly — with a short, ambiguous query, there is insufficient semantic signal to distinguish between the 50 candidate documents, so the reranker defaults to ranking by document length (longer documents first). The fix is query expansion using an LLM to rewrite short queries into more specific forms before retrieval.
- **C)** Cross-encoders perform poorly on short queries because they rely exclusively on BM25 keyword matching rather than semantic similarity; switching to a bi-encoder reranker resolves the short-query problem.
- **D)** Short queries produce empty vector embeddings in the Vector Search stage (vectors of all zeros), which causes all 50 retrieved candidates to have identical cosine similarity scores; the reranker then has no useful input and ranks randomly.

Correct Answer: B Explanation: B is correct. Cross-encoders jointly encode the query and each document together to produce a relevance score. With a rich, specific query ("SSL certificate validation error in Python 3.11 requests library"), the cross-encoder has abundant signal to distinguish highly relevant chunks from tangentially related ones. With a vague query ("error fix"), all 50 candidates may seem roughly equally relevant, making it hard for the cross-encoder to produce discriminative scores — resulting in near-arbitrary rankings. Query expansion (using an LLM to rewrite "error fix" into something like "common application error troubleshooting and debugging approaches") provides the cross-encoder with richer semantic signal. A is wrong because cross-encoders do not have minimum input length requirements and do not time out on short queries; the issue is semantic ambiguity, not a technical constraint. C is wrong because cross-encoders use neural attention mechanisms for semantic matching, not BM25 keyword matching; bi-encoders (not cross-encoders) are the vector-based approach. D is wrong because even a short query like "error fix" produces a meaningful (non-zero) vector embedding; embedding models do not produce all-zero vectors for valid text inputs. **Source:** Section 2: Data Preparation – Objective 8: Explain the role of re-ranking in the information retrieval process — docs.databricks.com (search: "Databricks Vector Search reranking" and "Foundation Model APIs cross-encoders")

Question 39

Difficulty: Proficiency

A developer prepares a chunking pipeline using `unstructured` to parse technical PDFs and classify elements as `Title`, `NarrativeText`, or `Table`. Their chunking strategy should preserve table integrity. When they implement the pipeline, they discover some tables are being split across chunks, resulting in partial table content in each chunk. What is the correct implementation fix?

- **A)** Set `max_characters=99999` in the `unstructured` chunking configuration, which forces all elements including tables to be treated as unsplittable atomic units regardless of size.
- **B)** Use `unstructured`'s `chunk_by_title()` function with `combine_text_under_n_chars` configured, and apply additional logic to detect `Table` elements and force them to be treated as unsplittable atomic chunks that are never split across boundaries.
- **C)** Convert all tables to images using a PDF rendering library before running `unstructured`, then use `pytesseract` to re-extract the table text — the OCR process naturally preserves table row integrity.
- **D)** Split the pipeline into two separate paths: one for `NarrativeText` using fixed-size chunking, and one for `Table` elements using document-aware chunking, then merge the resulting chunks back into a single `DataFrame` before embedding.

Correct Answer: B **Explanation:** B is correct. `unstructured`'s chunking functions allow you to specify element-type-aware behavior. By treating `Table` elements as atomic/unsplittable units (configuring the chunker to never split a single `Table` element across chunk boundaries), you preserve table integrity. This can be achieved by detecting `Table` elements before chunking and either (1) skipping them through the splitter and adding them as fixed chunks, or (2) configuring `unstructured`'s chunking parameters to respect element type boundaries. A is wrong because setting an extremely large `max_characters` value prevents any splitting but may create chunks vastly exceeding the embedding model's token limit for large tables or sections, causing different problems. C is wrong because converting existing tables to images and re-running OCR introduces accuracy loss and is computationally expensive; `unstructured` already parses table structure from PDFs without needing OCR. D is wrong because splitting into two separate pipelines based on element type is unnecessarily complex; `unstructured` is designed to handle multiple element types in a single unified pipeline with appropriate configuration. **Source:** Section 2: Data Preparation – Objective 3 & 7: Python packages and advanced chunking — docs.databricks.com (search: "RAG reference architecture data preparation" and "Advanced chunking strategies RAG")

Question 40

Difficulty: Proficiency

A data platform team ingests legal briefs (PDF) into a RAG pipeline. After 6 months in production, they discover: (1) older briefs from 2019–2020 contain outdated legal precedents that have since been overruled, but (2) the Vector Search index shows these old chunks as frequently retrieved because they are semantically similar to modern queries. What is the most comprehensive data governance solution?

- **A)** Delete the entire Vector Search index and rebuild it from scratch using only 2022–2025 documents, permanently excluding all pre-2021 content from the knowledge base to eliminate the stale precedent problem.
- **B)** Add a `document_year` metadata column to the source Delta table, apply an update to mark pre-2021 documents with a `is_superseded = true` flag, and configure the RAG chain to apply a metadata filter `is_superseded = false` at query time — excluding stale chunks from retrieval while preserving them in the Delta table for audit purposes.
- **C)** Fine-tune the embedding model on 2022–2025 legal briefs only so that pre-2021 documents produce embedding vectors that are dissimilar to modern queries, naturally reducing their retrieval frequency without requiring any metadata changes.
- **D)** Increase the Vector Search `num_results` from 5 to 50 and add a post-retrieval LLM filtering step that reads each chunk's embedded date metadata and removes any chunk dated before 2021 before passing context to the answer LLM.

Correct Answer: B **Explanation:** B is correct. This is a data governance and knowledge base lifecycle management challenge. Adding a `is_superseded` boolean flag to the source Delta table and filtering it out at query time is the correct approach because it: (1) preserves the historical data in Delta Lake for audit and compliance purposes, (2) ensures stale content never reaches the retrieval results, (3) is easily reversible (unsupersede a document by flipping the flag), and (4) works with Vector Search metadata filtering without requiring index rebuilds. A is wrong because permanently deleting data violates data retention requirements common in legal contexts, and the Vector Search index sync (CDF) would propagate the deletions to the index — but the underlying data would be lost. C is wrong because fine-tuning an embedding model to produce dissimilar vectors for specific documents is an extremely expensive and imprecise approach; embedding distance is not a reliable mechanism for enforcing business rules about document validity. D is wrong because adding an LLM filtering step for 50 retrieved chunks significantly increases latency and token cost, and relying on the LLM to filter by date is fragile and non-deterministic — metadata filtering at the Vector Search layer is more reliable and efficient. **Source:** Section 2: Data Preparation – Objective 5: Identify needed source documents — docs.databricks.com (search: "Generative AI data preparation best practices" and "Mosaic AI Vector Search index")

Question 41

Difficulty: Beginner

A developer needs to split a Markdown knowledge base article into chunks. The article uses `#` for the main title and `##` for major sections. They want each major section to be one chunk. Which chunking tool and split parameter should they use?

- **A)** Use Python's `str.split('\n\n')` to split on double newlines, which corresponds to paragraph boundaries in Markdown and keeps each paragraph as a separate chunk regardless of header level.
- **B)** Use LangChain's `MarkdownHeaderTextSplitter` configured to split on `##` headers, which produces one chunk per major section while keeping the section content together and adding the section title as metadata.
- **C)** Use `PyPDF2`'s page-based splitter to create one chunk per page, which aligns well with Markdown structure since each major section typically occupies a full rendered page.
- **D)** Use `pytesseract` to OCR-render the Markdown file as an image and then split on visual whitespace gaps between sections, which detects structural boundaries regardless of the underlying text format.

Correct Answer: B **Explanation:** B is correct. LangChain's `MarkdownHeaderTextSplitter` is designed exactly for this use case — it takes a list of header levels to split on (e.g., `[("##", "Section")]`) and produces one chunk per section, with the section title added as chunk metadata. This directly fulfills the requirement of one chunk per major section while preserving content integrity. A is wrong because double-newline splitting creates paragraph-level chunks, not section-level chunks — a major section with multiple paragraphs would be split into multiple chunks against the requirement. C is wrong because `PyPDF2` is a PDF parsing library, not a Markdown processing tool; Markdown files are plain text and do not have pages. D is wrong because `pytesseract` is an OCR tool for image-based text — applying it to a Markdown file (which is already machine-readable text) adds unnecessary complexity, accuracy loss, and computational overhead. **Source:** Section 2: Data Preparation – Objective 1: Apply a chunking strategy for a given document structure — [docs.databricks.com](https://docs.databricks.com/search/Data-preparation-for-RAG-applications) (search: "Data preparation for RAG applications")

Question 42

Difficulty: Beginner

A developer writes a chunk to a Unity Catalog Delta table with the column `chunk_text`. When they create a Databricks Vector Search index on this table, what additional column is automatically added to the index and what does it represent?

- **A)** A `chunk_summary` column is automatically added by Vector Search, which stores a 1-sentence LLM-generated summary of each chunk for fast keyword matching during retrieval.
- **B)** A `vector` or embedding column is automatically computed and stored by the Vector Search service — it contains the high-dimensional numerical array (embedding) of each `chunk_text`, enabling semantic similarity search.
- **C)** A `relevance_score` column is automatically added, containing a pre-computed relevance score between 0 and 1 that the index uses to rank chunks at query time without performing real-time vector computation.
- **D)** A `doc_hash` column is automatically added containing an MD5 hash of each `chunk_text`, which Vector Search uses as the primary deduplication key to prevent duplicate chunks from being stored in the index.

Correct Answer: B **Explanation:** B is correct. When a Databricks Vector Search index is created on a Delta table containing a text column, the Vector Search service automatically computes and stores the vector embedding for each row's text using the configured embedding model. This embedding column is the foundation of semantic similarity search — at query time, the query is also embedded and compared against stored embeddings using approximate nearest neighbor algorithms. A is wrong because Vector Search does not automatically generate LLM summaries of chunks; no LLM processing occurs at index creation time in standard configurations. C is wrong because there is no pre-computed static relevance score — relevance is computed dynamically at query time by comparing the query vector against stored chunk vectors, as relevance depends on the specific query. D is wrong because while `id` is used for deduplication and change tracking, Vector Search does not automatically add an MD5 hash column; deduplication is handled through the unique primary key column that the developer provides. **Source:** Section 2: Data Preparation – Objective 4: Define operations and sequence to write chunked text into Delta Lake — [docs.databricks.com](https://docs.databricks.com/search-and-indexing/vector-search.html) (search: "Create a Mosaic AI Vector Search index")

Question 43

Difficulty: Beginner

A developer has source documents as HTML files, PDFs, and plain `.txt` files. They want a unified extraction pipeline that handles all three formats without writing separate parsing code for each. Which library best satisfies this requirement?

- **A)** `PyPDF2` — because it is the most widely used document extraction library and implicitly supports HTML and TXT files as fallback formats when the primary PDF parsing mode does not detect PDF metadata.
- **B)** `beautifulsoup4` — because its HTML parser can handle any file extension by reading the raw bytes of PDF and TXT files as if they were HTML documents, producing consistent text output across all three formats.
- **C)** `unstructured` — because it is a multi-format document parsing library with native support for HTML, PDF, and plain text files, providing a unified API that auto-detects format and returns structured text elements.
- **D)** `pytesseract` — because it renders each file format to a JPEG image internally before performing OCR, producing consistent machine-readable text from any source format including HTML, PDF, and TXT.

Correct Answer: C **Explanation:** C is correct. `unstructured` is the only library among the options that natively handles all three formats (HTML, PDF, plain text) through a unified `partition()` function that auto-detects file format and routes to the appropriate parser. This eliminates the need to write separate parsing code for each format. A is wrong because `PyPDF2` is a PDF-specific library — it cannot parse HTML or TXT files; using it on non-PDF files produces errors or empty output. B is wrong because `beautifulsoup4` is an HTML/XML parser only — it cannot meaningfully parse binary PDF files; treating a PDF as HTML would produce garbled output from the binary data. D is wrong because `pytesseract` renders documents to images for OCR only when dealing with image-based content; applying OCR to digital HTML and TXT files (which already have machine-readable text) would be lossy, slow, and produce unnecessary accuracy errors. **Source:** Section 2: Data Preparation – Objective 3: Choose the appropriate Python package — [docs.databricks.com](https://docs.databricks.com/search/rag-reference-architecture-data-preparation) (search: "RAG reference architecture data preparation")

Question 44

Difficulty: Beginner

What is the purpose of the `chunk_overlap` parameter in a fixed-size chunking configuration?

- **A)** `chunk_overlap` controls how many chunks from adjacent documents are combined together into one super-chunk, enabling the retriever to always return multi-document context to the LLM.
- **B)** `chunk_overlap` specifies the number of tokens that are duplicated between consecutive chunks, preserving context at chunk boundaries so that sentences or ideas split by the chunk boundary appear in both adjacent chunks.
- **C)** `chunk_overlap` sets the minimum number of meaningful tokens a chunk must contain before it is written to the Delta table; chunks with fewer tokens than the overlap value are automatically discarded.
- **D)** `chunk_overlap` defines the maximum allowed similarity between two adjacent chunks — if their cosine similarity exceeds this value, one chunk is dropped to prevent the Vector Search index from storing near-duplicate embeddings.

Correct Answer: B **Explanation:** B is correct. `chunk_overlap` is the number of tokens shared between the end of one chunk and the beginning of the next chunk. For example, with `chunk_size=512` and `chunk_overlap=64`, the last 64 tokens of chunk N become the first 64 tokens of chunk N+1. This ensures that a sentence or concept that falls at the boundary between two chunks is fully represented in both, preventing context loss at chunk boundaries. A is wrong because `chunk_overlap` operates within a single document's chunking, not across different documents; it does not combine chunks from different documents. C is wrong because `chunk_overlap` is about boundary token duplication, not a minimum token threshold for filtering; a separate `min_chunk_size` parameter would handle filtering of small chunks. D is wrong because `chunk_overlap` is a text processing parameter set before embedding; it has nothing to do with cosine similarity or deduplication in the vector index. **Source:** Section 2: Data Preparation – Objective 1: Apply a chunking strategy — docs.databricks.com (search: "Data preparation for RAG applications")

Question 45

Difficulty: Beginner

A company stores its source documents in Databricks Volumes as PDF files. What is the correct pipeline sequence to make these documents searchable via a Databricks Vector Search index?

- **A)** Create the Vector Search index directly on the Volume path — Vector Search can automatically detect, parse, and embed PDF files stored in Volumes without any intermediate processing steps.
- **B)** Extract text from PDFs using a parsing library (e.g., `unstructured`), chunk the extracted text, write the chunks to a Unity Catalog Delta table with CDF enabled, then create the Vector Search index on the Delta table.
- **C)** Upload the PDF files to Databricks Model Serving as binary artifacts, which automatically vectorizes and indexes them — then configure the Vector Search index to query the Model Serving artifact store.
- **D)** Convert the PDFs to Delta tables using the Databricks Auto Loader PDF connector, which automatically detects PDF structure and creates one Delta row per paragraph, ready for immediate Vector Search indexing.

Correct Answer: B **Explanation:** B is correct. Databricks Vector Search requires a Delta Lake table as its data source — it cannot directly index files from Volumes. The correct pipeline is: (1) Use a parsing library (`unstructured`, `PyPDF2`, etc.) to extract text from PDFs in the Volume, (2) apply a chunking strategy to produce text chunks, (3) write chunks as rows to a Unity Catalog Delta table with CDF enabled, (4) create a Vector Search index on the Delta table. A is wrong because Vector Search does not have a native capability to directly read and parse PDF files from Volumes; it requires Delta tables as the source. C is wrong because Databricks Model Serving is for deploying ML models as REST APIs — it is not an artifact indexing service and has no automatic vectorization pipeline for documents. D is wrong because Auto Loader is designed to ingest structured and semi-structured data (CSV, JSON, Parquet, Avro) — there is no built-in PDF connector that automatically converts PDFs to paragraph-level Delta rows. **Source:** Section 2: Data Preparation – Objective 4: Define operations and sequence to write chunked text into Delta Lake — docs.databricks.com (search: "Create a Mosaic AI Vector Search index")

Question 46

Difficulty: Intermediate

A developer discovers that their RAG chatbot is returning wrong answers specifically for topics that span multiple sections of a long document. For example, "What are the steps to set up SSO?" returns incomplete steps because step 1 is in the "Prerequisites" section and steps 2–5 are in the "Configuration" section — split across multiple chunks. What chunking strategy best addresses this multi-section problem?

- **A)** Use smaller chunk sizes (e.g., 100 tokens) so that more chunks are retrieved in the top-k, increasing the probability that all prerequisite and configuration section chunks appear in the retrieved set simultaneously.
- **B)** Use document-aware chunking at a higher structural level (e.g., split only at the top-level `#` document header instead of `##` section headers), so that all sections of a single document remain in one larger chunk that the LLM can read end-to-end.
- **C)** Use a sliding window approach where every chunk overlaps by 90% of its length with the previous chunk, effectively creating near-duplicate chunks for every token position to ensure no cross-section content is missed.
- **D)** Add a `related_sections` metadata column to each chunk that lists the section IDs of adjacent sections, and modify the retriever to automatically fetch adjacent sections when any section chunk is retrieved.

Correct Answer: B **Explanation:** B is correct. When topics span multiple document sections, splitting at a lower granularity (e.g., `##` headers) creates inter-section dependency that no retriever can reliably bridge. The fix is to chunk at a higher structural level — splitting only on the document title (`#`) keeps all sections of one document together in a single (larger) chunk. The trade-off is larger chunks with more mixed content, but for documents where cross-section queries are common, it is the correct architectural choice. This may require combining with a size-limiting second split if the document exceeds the token limit. A is wrong because using smaller chunks (100 tokens) creates more fragmented chunks with even less context per chunk — more fragments does not solve the cross-section dependency problem. C is wrong because 90% overlap creates massive data redundancy (a 512-token chunk with 90% overlap produces chunks every 51 tokens), exploding storage and index size without meaningfully improving the multi-section retrieval problem. D is wrong because adding `related_sections` metadata and modifying the retriever requires significant custom infrastructure code and is fragile — the prerequisite for this approach is accurate section relationship metadata, which is itself complex to maintain.

Source: Section 2: Data Preparation – Objective 7: Design retrieval systems using advanced chunking strategies — docs.databricks.com (search: "Advanced chunking strategies RAG")

Question 47

Difficulty: Intermediate

A developer runs the following code to prepare a Vector Search source table:

```
spark.sql("""
  CREATE TABLE IF NOT EXISTS main.docs.chunks
  (id BIGINT, chunk_text STRING)
  TBLPROPERTIES (delta.enableChangeDataFeed = true)
""")
```

Is this approach correct for enabling CDF? Are there any risks with the `id BIGINT` column type?

- **A)** The code is incorrect — CDF must be enabled separately with `ALTER TABLE` after table creation; including it in `CREATE TABLE TBLPROPERTIES` silently fails and CDF is not actually enabled.
- **B)** The code is correct — CDF can be enabled at table creation time via `TBLPROPERTIES`. However, `id BIGINT` risks column type conflicts if the source system generates string-format IDs (e.g., UUIDs), which must be cast to BIGINT or the pipeline will fail with type errors.
- **C)** The code is correct and the `id BIGINT` type is always appropriate since Vector Search requires integer primary keys; string-type IDs are not supported as primary key columns for Vector Search indexes.
- **D)** The code is incorrect — CDF requires the source table to use Delta Lake format v3 or higher; specifying `TBLPROPERTIES` at creation time only works with format v3, and Unity Catalog tables default to format v1.

Correct Answer: B **Explanation:** B is correct. Including `delta.enableChangeDataFeed = true` in the `TBLPROPERTIES` at `CREATE TABLE` time is a fully valid and supported approach — CDF is activated from the first write. However, `id BIGINT` is a risk: if source document IDs are UUID strings (e.g., "a1b2c3d4-..."), casting them to BIGINT would fail or produce incorrect values. The developer should use `id STRING` if the source IDs are not guaranteed to be integers. Databricks Vector Search supports both string and integer primary key column types. A is wrong because CDF can be enabled either at creation time (via `TBLPROPERTIES` in `CREATE TABLE`) or after creation (via `ALTER TABLE`); both approaches are valid and supported. C is wrong because Databricks Vector Search supports string-type primary key columns — BIGINT is not the only accepted type. D is wrong because there is no "format version" prerequisite for CDF; CDF has been available in Delta Lake for years and works with the default table format in Unity Catalog. **Source:** Section 2: Data Preparation – Objective 4: Define operations and sequence to write chunked text into Delta Lake — docs.databricks.com (search: "Enable Change Data Feed" and "Create a Mosaic AI Vector Search index")

Question 48

Difficulty: Intermediate

A team is evaluating whether to use a cross-encoder reranker or a bi-encoder re-scoring step as stage 2 after Vector Search retrieval. The application serves 500 queries per second. Which technical trade-off determines the correct choice?

- **A)** Cross-encoders produce more accurate relevance scores but are computationally expensive (they process each query-document pair together); bi-encoder re-scoring is faster but less accurate (it scores query and documents independently). For 500 QPS, the latency cost of a cross-encoder on 50 candidates per query must be evaluated against the quality improvement.
- **B)** Cross-encoders are always faster than bi-encoders at scale because they use precomputed document representations; bi-encoders recompute document embeddings at query time, making them slower for high-volume applications.
- **C)** Bi-encoders produce more accurate reranking than cross-encoders at high query volumes because bi-encoders can be parallelized across GPU cores, while cross-encoders run sequentially on CPU only.
- **D)** Cross-encoders and bi-encoders produce identical reranking quality for queries longer than 50 tokens; the choice only matters for very short queries (under 10 tokens) where cross-encoders have a slight advantage.

Correct Answer: A **Explanation:** A is correct. This is the fundamental cross-encoder vs. bi-encoder trade-off: cross-encoders jointly encode query and document together, giving them access to fine-grained query-document interactions for high accuracy, but at the cost of running a separate forward pass for each candidate pair ($O(n)$ compute per query). Bi-encoders encode query and documents separately (documents can be pre-encoded) and score by dot product — much faster but less precise because the joint attention mechanism is absent. At 500 QPS with 50 candidates each, a cross-encoder must perform 25,000 inference calls per second, which may exceed latency targets. The team must benchmark latency and quality for their specific throughput requirement. B is wrong because this reverses the trade-off: bi-encoders use precomputed document embeddings (stored in the index) and are faster; cross-encoders process query-document pairs jointly and are computationally more expensive. C is wrong because accuracy trade-off is the opposite — cross-encoders are more accurate than bi-encoder re-scoring because of their joint attention mechanism; accuracy claims in C are incorrect. D is wrong because the accuracy gap between cross-encoders and bi-encoders is not determined by query token length; cross-encoders consistently outperform bi-encoders across all query lengths. **Source:** Section 2: Data Preparation – Objective 8: Explain the role of re-ranking — docs.databricks.com (search: "Databricks Vector Search reranking" and "Foundation Model APIs cross-encoders")

Question 49

Difficulty: Intermediate

A developer adds document metadata (title, author, publication_date) as additional columns to the source Delta table used by Vector Search. A colleague asks: "Do these metadata columns affect the embedding computation?" What is the technically correct answer?

- **A)** Yes — Vector Search automatically concatenates all string columns from the Delta table into the embedding input, so adding title and author columns increases the semantic richness of the embedding vector.
- **B)** No — the embedding is computed only from the designated `chunk_text` column specified when creating the index. Metadata columns (title, author, publication_date) are stored in the index for filtering and retrieval but do not influence the embedding computation.
- **C)** Yes — but only the `title` column is automatically appended to `chunk_text` before embedding; other metadata columns (author, publication_date) are stored separately without affecting the embedding.
- **D)** No — metadata columns are not stored in the Vector Search index at all; they remain only in the source Delta table and must be joined back after retrieval using the `id` column to access metadata in the application.

Correct Answer: B **Explanation:** B is correct. When creating a Databricks Vector Search index, you specify which column contains the text to embed (e.g., `source_column="chunk_text"`). Only this designated column's text is passed to the embedding model. Metadata columns (title, author, date) are stored alongside the embedding in the index for use as retrieval filters (e.g., `filters_json='{ "author": "Jane Smith" }'`) or as returned metadata in results, but they do not contribute to the embedding vector itself. A is wrong because Vector Search does not automatically concatenate all string columns — only the explicitly designated `source_column` is embedded. Adding metadata columns to the table does not change what gets embedded. C is wrong because there is no automatic title-appending behavior; embedding input is strictly limited to the designated source column. D is wrong because metadata columns ARE stored in the Vector Search index alongside the embeddings — this is how metadata filtering works at query time without requiring a separate Delta table join. **Source:** Section 2: Data Preparation – Objective 4: Define operations and sequence to write chunked text into Delta Lake — docs.databricks.com (search: "Create a Mosaic AI Vector Search index")

Question 50

Difficulty: Intermediate

A developer is evaluating retrieval quality using a 200-question test set. For each question, they know the exact chunk IDs that should be returned. They want to measure both "are the right chunks being retrieved?" AND "are they being returned in the right order?" Which combination of metrics should they compute?

- **A)** `Precision@k` only — it captures both retrieval accuracy (whether right chunks are returned) and ranking quality (whether they are returned in the right order) in a single unified metric.
- **B)** `Recall@k` and `Precision@k` together — `Recall@k` measures whether all relevant chunks are found, and `Precision@k` measures what fraction of returned chunks are relevant; together they capture coverage and accuracy but not ranking.
- **C)** `Recall@k` and `nDCG@k` together — `Recall@k` measures whether all relevant chunks are found (coverage), and `nDCG@k` measures position-weighted relevance quality (ranking) — together they capture both retrieval completeness and ranking quality.
- **D)** `MRR` (Mean Reciprocal Rank) only — it directly measures both the fraction of correctly retrieved chunks AND their rank positions, making it the single comprehensive metric for evaluating both retrieval and ranking simultaneously.

Correct Answer: C **Explanation:** C is correct. The question requires two distinct measurements: (1) "Are the right chunks being retrieved?" is answered by `Recall@k` — it measures coverage (what fraction of known-relevant chunks appear in the top-k). (2) "Are they being returned in the right order?" is answered by `nDCG@k` — it measures position-weighted relevance (giving more credit to relevant chunks at higher ranks). Together, `Recall@k` and `nDCG@k` comprehensively cover both retrieval completeness and ranking quality. A is wrong because `Precision@k` measures what fraction of returned chunks are relevant (accuracy) but does not measure ranking quality — a `Precision@5` of 0.8 is the same whether the 4 relevant chunks are at ranks 1–4 or ranks 2–5. B is wrong because while `Recall@k` and `Precision@k` together capture coverage and accuracy, they do not capture ranking quality — as noted, neither metric distinguishes whether relevant results appear at rank 1 or rank 10. D is wrong because `MRR` (Mean Reciprocal Rank) measures the rank of the FIRST relevant result only — it does not measure coverage (`Recall`) or the ordering of ALL relevant results. **Source:** Section 2: Data Preparation – Objective 6: Use tools and metrics to evaluate retrieval performance — docs.databricks.com (search: "MLflow evaluate for RAG")

Question 51

Difficulty: Advanced

A developer instructs a junior engineer: "Add `@mlflow.trace(span_type='RETRIEVER')` to the retrieval function before the evaluation run." The junior engineer adds it to the wrong function — they add it to the LLM call wrapper instead of the retrieval function. What will happen when `mlflow.genai.evaluate()` runs the `RetrievalRelevance` scorer?

- **A)** The scorer will raise a `SpanTypeConflict` exception because the `RETRIEVER` span type is exclusively reserved for Vector Search function calls; applying it to any other function violates MLflow's span type schema validation.
- **B)** The scorer will not find a `RETRIEVER`-typed span in the expected location of the trace tree; it will look for retrieved documents in the span output and instead find the LLM's generated text, causing the scorer to fail or return null/nonsensical scores.
- **C)** The scorer will automatically detect that the decorated function is an LLM call (based on the input/output format) and switch to using `AnswerRelevance` scoring instead of `RetrievalRelevance` scoring, adapting to the span content.
- **D)** The scorer will work correctly because MLflow's `RetrievalRelevance` scorer reads all spans in the trace tree regardless of their type, averaging relevance scores across all captured inputs and outputs.

Correct Answer: B Explanation: B is correct. MLflow's `RetrievalRelevance` scorer specifically looks for a span typed as `"RETRIEVER"` in the trace tree to find the retrieval inputs (the query) and outputs (the retrieved document chunks). If the `RETRIEVER` span type is applied to the LLM call wrapper instead, the span that MLflow finds in the `RETRIEVER` slot will contain the LLM's generated text as its output rather than document chunks. The scorer will either fail to parse the LLM output as retrieved documents, return null scores, or produce meaningless scores based on misidentified data. A is wrong because MLflow does not enforce which Python function a span type can be applied to — the `RETRIEVER` type is a semantic label, not a technical enforcement mechanism; misapplying it doesn't raise an exception. C is wrong because MLflow evaluation scorers do not automatically detect function types or switch scorer logic based on span content; they operate on whatever data is in the specified span. D is wrong because `RetrievalRelevance` specifically requires a `RETRIEVER`-typed span; it does not aggregate across all span types in the trace. **Source:** Section 2: Data Preparation – Objective 6: Use tools and metrics to evaluate retrieval performance — docs.databricks.com (search: "MLflow Tracing")

Question 52

Difficulty: Advanced

A production RAG pipeline uses fixed-size chunking with 512 tokens and retrieves top-5 chunks. Engineers discover that for complex, multi-part user queries (e.g., "Compare the warranty terms, return policy, and price-matching guarantee for Product X"), individual chunks only cover one aspect each, causing the LLM to receive incomplete context. They decide to increase `num_results` from 5 to 15 to capture all three aspects. What new problem does this introduce, and what is the more architecturally sound fix?

- **A)** Increasing `num_results` to 15 causes the Vector Search index query latency to increase by 3×, making the pipeline too slow for real-time use; the fix is to use a faster approximate nearest neighbor algorithm instead.
- **B)** Increasing `num_results` to 15 sends 15 chunks (potentially 7,680 tokens) to the LLM, which may exceed the LLM's context window limit and dilute the LLM's attention across more content — the sound fix is multi-query decomposition: split the compound question into 3 sub-queries, retrieve top-5 for each, and deduplicate before assembling context.
- **C)** Increasing `num_results` to 15 violates the Databricks Vector Search maximum results limit of 10 per query; the fix is to run two separate Vector Search queries with `num_results=7` and `num_results=8` and merge the results.
- **D)** Increasing `num_results` to 15 causes the MLflow `RetrievalRelevance` scorer to time out because it must run an LLM judge 15 times instead of 5 times per query, making evaluation computationally infeasible for production use.

Correct Answer: B **Explanation:** B is correct. Increasing `num_results` to 15 passes significantly more tokens (up to $15 \times \sim 512 = 7,680$ tokens) to the LLM. Modern LLMs have context window limits (even if large), and more content can dilute the LLM's attention, leading to reduced answer quality due to the "lost in the middle" phenomenon. The architecturally sound fix for multi-part queries is query decomposition: break "Compare warranty, return policy, and price-matching for Product X" into three focused sub-queries, retrieve top-5 for each, deduplicate overlapping chunks, and assemble a focused context set. This retrieves relevant context for each aspect without inflating the total context size unnecessarily. A is wrong because Databricks Vector Search is highly optimized for fast approximate nearest neighbor search; increasing `num_results` from 5 to 15 has minimal impact on query latency — the dominant latency factors are network and embedding computation. C is wrong because Databricks Vector Search supports retrieving significantly more than 10 results per query — there is no 10-result limit. D is wrong because evaluation run time is a development/testing concern, not a production concern; even if evaluation takes longer, it does not affect the production pipeline's correctness. **Source:** Section 2: Data Preparation – Objective 7: Design retrieval systems using advanced chunking strategies — docs.databricks.com (search: "Advanced chunking strategies RAG")

Question 53

Difficulty: Advanced

A developer's reranking stage uses a cross-encoder model to score 50 candidate chunks. The cross-encoder is deployed on a Databricks Model Serving CPU endpoint. Average reranker latency is 2.8 seconds per query. A performance engineer proposes: "We should use the cross-encoder's output scores as permanent metadata fields in the Vector Search index, so we don't need to run the reranker at query time." What is technically wrong with this proposal?

- **A)** Storing pre-computed reranker scores in the Vector Search index is a valid optimization; the engineer's proposal is correct and would eliminate all reranker latency at query time.
- **B)** Pre-computed cross-encoder scores are query-independent (computed once at index creation time), but cross-encoders are designed to score query-document relevance jointly — the score is specific to a particular query. A score computed during index creation is meaningless for a different user query at runtime.
- **C)** Pre-computed scores would only work if stored in the Delta source table first, then synced to Vector Search; storing scores directly in the Vector Search index without going through the Delta source table violates the CDF sync architecture.
- **D)** The proposal fails because Databricks Vector Search metadata fields can only store string or integer values; floating-point relevance scores from a cross-encoder cannot be stored as index metadata.

Correct Answer: B **Explanation:** B is correct. This is the fundamental flaw in the proposal. A cross-encoder produces a relevance score for a specific (query, document) pair — the score changes with every different query. Pre-computing a score at index creation time would require knowing the query in advance, which is impossible for a live system serving diverse user queries. The scores computed at index time (without knowing the user's query) would be meaningless approximations. The reranker's value IS the query-dependent re-scoring — pre-computing it eliminates exactly the quality benefit it provides. A is wrong because the proposal is technically flawed for the reason explained in B; it is not a valid optimization. C is wrong because while CDF is required for Delta-to-VectorSearch sync, the proposal's flaw is not about the data pipeline architecture — it is about the fundamental query-dependence of cross-encoder scores. D is wrong because Databricks Vector Search metadata supports numeric (float) types; the proposal's flaw is not about data type limitations. **Source:** Section 2: Data Preparation – Objective 8: Explain the role of re-ranking — docs.databricks.com (search: "Databricks Vector Search reranking" and "Foundation Model APIs cross-encoders")

Question 54

Difficulty: Advanced

A team discovers that their RAG pipeline has high `Precision@5` (0.91) but very low `Recall@5` (0.28). They have 8 million chunks in the Vector Search index covering 200,000 source documents. What is the most likely architectural cause, and what change addresses it?

- **A)** The embedding model is too large (7B parameters), making it overfit to training data and retrieve only chunks it "memorizes" from training — switching to a smaller 125M parameter embedding model improves generalization and recall.
- **B)** The `num_results=5` setting is too low for a knowledge base of 8 million chunks — there may be 15–20 relevant chunks for complex queries, but only 5 are returned. Increasing `num_results` to 20–50, adding a reranker to select the best 5, improves recall while preserving precision.
- **C)** The Delta source table uses partitioning by document ID, which causes Vector Search to only search within the partition matching the query's document ID — adding an `id` metadata filter to query all partitions simultaneously fixes the recall problem.
- **D)** The Vector Search index sync is in `TRIGGERED` mode with manual syncs every 30 days — 72% of relevant chunks were added in the last 30 days and are not yet indexed. Switching to `CONTINUOUS` sync immediately restores full recall.

Correct Answer: B **Explanation:** B is correct. `Recall@5 = 0.28` with `Precision@5 = 0.91` indicates that the 5 retrieved chunks are relevant (good precision) but many other relevant chunks are missed (low recall). With 8 million chunks and complex queries potentially having 15–20 truly relevant chunks, retrieving only 5 means the system misses most of them. The architectural fix is to increase `num_results` to 20–50 (a larger retrieval pool that has a higher probability of containing all relevant chunks), then apply a reranker to select the best 5 for the LLM — maintaining precision while dramatically improving recall. A is wrong because embedding model size does not directly cause low recall; a 7B parameter model is not more "memorized" than a smaller model in a way that would explain low recall on a private knowledge base. C is wrong because Vector Search performs global approximate nearest neighbor search across the entire index — it does not partition searches by document ID. D is wrong because if chunks are missing from the index due to sync lag, `Precision@5` would also be low (wrong chunks would be retrieved) — the fact that precision is high (91%) confirms that indexed chunks are correct; the problem is retrieval coverage, not missing data. **Source:** Section 2: Data Preparation – Objective 6 & 8: Retrieval metrics and re-ranking — docs.databricks.com (search: "MLflow evaluate for RAG" and "Databricks Vector Search reranking")

Question 55

Difficulty: Advanced

A developer applies `unstructured`'s `partition_pdf()` to a financial report PDF and observes that numeric data from tables is extracted as `NarrativeText` elements instead of `Table` elements. This causes table rows to be chunked as prose sentences, losing the tabular structure. What is the root cause and the fix?

- **A)** The financial report PDF uses non-standard fonts for table cells, which causes `unstructured`'s table detector to misclassify the cells as prose text. The fix is to convert the PDF to a standard font using a PDF editing tool before running `partition_pdf()`.
- **B)** The `partition_pdf()` function defaults to text-layer extraction without table structure detection; enabling the table inference mode (e.g., `infer_table_structure=True` and using the `hi_res` strategy) instructs `unstructured` to use a table detection model to correctly classify and extract tabular elements.
- **C)** `unstructured` cannot detect tables in financial PDFs because financial tables use merged cells and multi-column headers; the only fix is to manually export the PDF tables to CSV files and ingest them through a separate pipeline.
- **D)** The PDF tables are using a `<div>`-based HTML table simulation instead of standard PDF table structures; switching to `beautifulsoup4` as the parser (which understands `<div>` layouts) correctly identifies and extracts the table data.

Correct Answer: B **Explanation:** B is correct. `unstructured`'s `partition_pdf()` function operates in two modes: a fast default mode (`strategy="fast"`) that uses the PDF text layer for extraction and may not detect table structures correctly, and a high-resolution mode (`strategy="hi_res"` , `infer_table_structure=True`) that uses a computer vision-based table detection model (e.g., a YOLO-based table detector) to identify table regions and extract them as structured `Table` elements. Enabling `hi_res` mode with table inference is the correct fix. A is wrong because font type does not affect `unstructured`'s table detection — table detection in `hi_res` mode uses visual layout analysis, not font metadata. C is wrong because `unstructured` with `hi_res` mode and `infer_table_structure=True` is designed to handle complex financial tables with merged cells; manual CSV export adds unnecessary manual labor. D is wrong because PDF files do not contain HTML `<div>` elements — PDFs use a completely different internal format (PostScript-based), and `beautifulsoup4` cannot parse PDF binary content. **Source:** Section 2: Data Preparation – Objective 3: Choose the appropriate Python package — docs.databricks.com (search: "RAG reference architecture data preparation")

Question 56

Difficulty: Proficiency

A data engineering team is building a production RAG data pipeline using Delta Live Tables (DLT). The pipeline must: (1) parse PDF documents from a Volume, (2) clean and filter content, (3) chunk text, and (4) write chunks to a Delta table with CDF enabled for Vector Search sync. What is the correct DLT implementation consideration that differs from a standard notebook pipeline?

- **A)** DLT pipelines automatically enable Change Data Feed on all output Delta tables they create; the developer does not need to run `ALTER TABLE ... SET TBLPROPERTIES (delta.enableChangeDataFeed = true)` — DLT manages CDF automatically.
- **B)** DLT pipelines cannot write to Unity Catalog Delta tables; the chunked output table must be written to the Hive metastore, and a separate job must copy it to Unity Catalog before Vector Search can index it.
- **C)** DLT requires all Python transformation logic (PDF parsing, chunking) to be implemented using only Spark DataFrame operations; using non-Spark Python libraries like `unstructured` or `PyPDF2` inside DLT Python decorators is not supported.
- **D)** DLT streaming tables cannot be configured with `TBLPROPERTIES` at table definition time; CDF must be enabled with `ALTER TABLE` after the first pipeline run completes and the table is physically created.

Correct Answer: A **Explanation:** A is correct. One of Delta Live Tables' key features for streaming use cases is that it automatically enables Change Data Feed on all managed output tables. This is because DLT is inherently incremental and uses CDF internally for its own incremental processing. For Vector Search integration, this means DLT output tables are CDF-ready by default — a significant advantage over standard notebook pipelines that require the developer to manually enable CDF with `ALTER TABLE`. B is wrong because DLT fully supports Unity Catalog as the output metastore; writing to `catalog.schema.table` with `@dlt.table(name="catalog.schema.chunks")` is supported and recommended. C is wrong because DLT Python pipelines support arbitrary Python code within `@dlt.table` decorated functions, including non-Spark libraries like `unstructured` and `PyPDF2` via UDFs or Spark's `mapInPandas`; the only requirement is that the function returns a Spark DataFrame. D is wrong because DLT streaming tables do support `TBLPROPERTIES` at definition time via the `@dlt.table(table_properties={"delta.enableChangeDataFeed": "true"})` decorator argument — though as noted in A, DLT also enables CDF automatically. **Source:** Section 2: Data Preparation – Objective 4: Define operations and sequence to write chunked text into Delta Lake — docs.databricks.com (search: "Create a Mosaic AI Vector Search index" and "Enable Change Data Feed")

Question 57

Difficulty: Proficiency

An organization's RAG pipeline serves a global team, and source documents are in English, French, German, and Japanese. The Vector Search index uses the Databricks `databricks-bge-large-en` embedding model. Users report poor retrieval for French, German, and Japanese queries even when relevant documents exist in those languages. What is the complete diagnosis and remediation?

- **A)** The `databricks-bge-large-en` model is English-only; it produces low-quality embeddings for French, German, and Japanese text because it was trained on English data. The fix is to replace it with a multilingual embedding model (e.g., `text-embedding-3-large` or `paraphrase-multilingual-mpnet-base-v2`) that produces semantically aligned cross-lingual embeddings.
- **B)** The retrieval failure is caused by the Delta table character encoding — French, German, and Japanese characters are stored as UTF-16 in the Delta table, but the embedding model requires UTF-8; the fix is to convert the Delta table's encoding to UTF-8.
- **C)** The multilingual retrieval failure is caused by the Vector Search index's approximate nearest neighbor algorithm (HNSW) which can only handle Latin character embeddings; switching to an exact nearest neighbor search resolves the language limitation.
- **D)** The fix is to add a language detection column to each chunk and apply a metadata filter at query time that restricts retrieval to chunks in the user's detected language, eliminating cross-language retrieval issues.

Correct Answer: A **Explanation:** A is correct. The `databricks-bge-large-en` model (BGE Large English) is trained on English text and produces high-quality embeddings for English. Its embedding space is not aligned with French, German, or Japanese text — French/German/Japanese queries will not produce embedding vectors that are close to the corresponding French/German/Japanese document embeddings, causing retrieval failure. The fix is a multilingual embedding model that maps text in different languages to a shared semantic embedding space, enabling cross-lingual retrieval (English query → French document, etc.). B is wrong because Delta Lake stores strings as UTF-8 by default in Parquet files, and embedding models accept Python strings (already decoded from UTF-8); encoding is not the source of multilingual retrieval failure. C is wrong because the approximate nearest neighbor algorithm (HNSW or similar) operates on vectors of floating-point numbers — it has no awareness of the original character encoding or language; the language problem is in the embedding model, not the search algorithm. D is wrong because language-based metadata filtering restricts users to retrieving documents in their own language — this prevents cross-lingual retrieval (an English query finding a French document about the same topic) and does not address the core embedding quality issue. **Source:** Section 2: Data Preparation – Objective 1 & 4: Chunking constraints and Delta Lake setup — docs.databricks.com (search: "Create a Mosaic AI Vector Search index" and "Data preparation for RAG applications")

Question 58

Difficulty: Proficiency

A developer is designing the chunking strategy for a knowledge base of 10,000 API reference documents. Each document has a fixed structure: **Overview** (100–200 tokens), **Parameters** (50–500 tokens depending on API complexity), **Examples** (200–800 tokens), and **Error Codes** (100–300 tokens). Users query specific sections (e.g., "What parameters does the /users endpoint accept?"). What chunking design provides optimal retrieval precision?

- **A)** Apply fixed-size 512-token chunking across all sections of all documents, treating the entire document as flat text and relying on the embedding model to implicitly learn the section structure from repeated section header tokens.
- **B)** Apply section-aware chunking: use the document's section headers as split boundaries, creating separate chunks for **Overview**, **Parameters**, **Examples**, and **Error Codes** — each chunk prefixed with **[Document: /users API] [Section: Parameters]** metadata to enable targeted retrieval.
- **C)** Apply Parent-Child chunking where each full API document is the parent and each token within the document is the child chunk, enabling the retrieval of the exact token that matches the query and providing the full document as LLM context.
- **D)** Apply no chunking — store each entire API reference document as one chunk per document, relying on the LLM's 128K context window to process all 10,000 documents simultaneously for each user query.

Correct Answer: B **Explanation:** B is correct. Given the consistent, predictable section structure of API reference documents and the highly targeted nature of user queries (users ask about specific sections, not entire documents), section-aware chunking is optimal. Each section becomes its own chunk, and the metadata prefix (**[Document: /users API] [Section: Parameters]**) enriches the embedding with contextual information that helps the retriever match section-specific queries. A query for "parameters of /users endpoint" will have high cosine similarity to the Parameters section chunk with its metadata prefix, producing highly precise retrieval. A is wrong because fixed-size chunking ignores the document structure and mixes section content across chunks, reducing precision for section-specific queries. C is wrong because using individual tokens as child chunks is computationally absurd and semantically meaningless; child chunks in Parent-Child should be semantically coherent (sentences or paragraphs), not individual tokens. D is wrong because Vector Search requires chunked text as rows in a Delta table; storing an entire document as one row would (a) likely exceed the embedding model's token limit and (b) reduce retrieval precision since the entire document's content is condensed into one embedding. **Source:** Section 2: Data Preparation – Objective 1 & 7: Chunking strategy and advanced chunking — docs.databricks.com (search: "Advanced chunking strategies RAG")

Question 59

Difficulty: Proficiency

A RAG pipeline serves a compliance team that queries a knowledge base of 500,000 regulatory documents. A compliance officer reports: "The chatbot sometimes references documents that were superseded by newer regulations — it mixes old and new guidance." Retrieval metrics show `Precision@5 = 0.88` and `Recall@5 = 0.79`. What is the correct diagnosis — is this a retrieval quality problem or a data governance problem — and what is the right fix?

- **A)** This is a retrieval quality problem — `Precision@5 = 0.88` is below the 0.95 threshold required for regulatory use cases; improving the embedding model and adding a cross-encoder reranker will prevent outdated documents from being retrieved.
- **B)** This is a data governance problem, not a retrieval quality problem — the retrieval metrics are healthy (88% precision, 79% recall), meaning the retriever correctly finds the most semantically similar documents. The issue is that superseded documents are still present and active in the knowledge base. The fix is a data lifecycle process: add a `status` column (`active` / `superseded`) to the source Delta table, update superseded documents to `status='superseded'`, and apply a metadata filter `status='active'` at query time.
- **C)** This is a retrieval quality problem — `Recall@5 = 0.79` means 21% of relevant regulatory documents are missed per query; improving `num_results` to 25 and adding a reranker will surface all active regulations including the most recent ones.
- **D)** This is a data governance problem that requires re-training the embedding model on only the current, active regulatory documents, so that embeddings for superseded documents become dissimilar to user queries and are naturally deprioritized by Vector Search.

Correct Answer: B **Explanation:** B is correct. The retrieval metrics (`Precision@5 = 0.88`, `Recall@5 = 0.79`) indicate the retrieval system is performing well — it is correctly surfacing the most semantically similar documents. The problem is that "semantically similar" includes superseded documents that are topically related to the query (they cover the same regulation) but contain outdated guidance. This is not a retrieval failure — it is a data governance failure (outdated documents should not be in the active knowledge base). The correct fix is a data lifecycle management process: mark superseded documents with a `status='superseded'` flag and filter them out at query time. A is wrong because 0.88 precision is not the cause of the compliance issue — even perfect precision of 1.0 would not prevent semantically relevant but superseded documents from being retrieved if they remain active in the index. C is wrong for the same reason as A — improving recall retrieves more superseded documents, making the problem worse. D is wrong because re-training the embedding model is expensive and imprecise; metadata filtering is the correct, deterministic, and maintainable governance solution. **Source:** Section 2: Data Preparation – Objective 5 & 6: Identify source documents and evaluate retrieval — docs.databricks.com (search: "Generative AI data preparation best practices" and "Mosaic AI Vector Search index")

Question 60

Difficulty: Proficiency

A team is building a RAG pipeline for 200,000 scientific papers in biology. Each paper averages 15,000 tokens. They use Parent-Child chunking: each paper's abstract (200–500 tokens) as child chunks for vector search, with the full paper as parent context. A researcher reports: "When I ask detailed methodology questions, the answers are missing key experimental details." Investigation confirms the relevant methodology section exists in the full paper (parent) but is 4,000–6,000 tokens into the paper. What is the flaw in this design, and what is the fix?

- **A)** The flaw is that the abstract (child chunk) correctly identifies the paper but the full paper (parent, ~15,000 tokens) exceeds most LLMs' useful context window — the LLM receives the full paper but loses the methodology section buried at 4,000–6,000 tokens in due to the "lost in the middle" phenomenon. The fix is to add methodology section chunks as additional child chunks, enabling direct retrieval of the methodology section as a focused child with the surrounding section (not full paper) as parent.
- **B)** The flaw is that biology papers use domain-specific terminology that the embedding model cannot encode accurately; the abstract's general language creates embedding vectors that don't match methodology-specific queries. The fix is to fine-tune the embedding model on biology paper abstracts.
- **C)** The flaw is that parent chunks (full papers at 15,000 tokens) exceed the Vector Search maximum document size of 8,192 tokens per chunk; storing full papers in Vector Search raises an exception. The fix is to limit parent chunks to 8,192 tokens by truncating papers at that limit.
- **D)** The flaw is that the methodology section requires structured tabular data that the LLM cannot process from plain text; converting methodology sections to JSON structured format before storing in the Delta table enables the LLM to correctly interpret experimental parameters.

Correct Answer: A **Explanation:** A is correct. This is a nuanced Parent-Child design flaw. The abstract correctly retrieves the right paper (high recall at the paper level) but the parent context (full 15,000-token paper) overwhelms the LLM — methodology details buried at positions 4,000–6,000 in a 15,000-token paper are subject to the "lost in the middle" phenomenon where LLMs attend less to content in the middle of very long contexts. The fix is hierarchical chunking with multiple child types: embed both the abstract AND the individual section chunks (abstract, introduction, methods, results, conclusion) as children. A methodology query then retrieves the methods child chunk directly, and its parent context is the methods section (~1,500–2,000 tokens) rather than the entire 15,000-token paper — giving the LLM focused, relevant context. B is wrong because while domain-specific terminology is a real concern, the described problem (methodology details in the parent paper not being surfaced) is a context position problem, not an embedding model vocabulary problem. C is wrong because the parent chunks (full papers) are stored in the Delta table, not in Vector Search — only the child embeddings (abstracts) are stored as vectors in the Vector Search index; full paper text length does not have a Vector Search size limit. D is wrong because the issue is context window positioning, not data format; biology methodology sections in plain text are fully processable by LLMs — converting them to JSON adds structure that the LLM doesn't need to understand the content. **Source:** Section 2: Data Preparation – Objective 7: Design retrieval systems using advanced chunking strategies — docs.databricks.com (search: "Advanced chunking strategies RAG")